数据库原理

Database Principle

陶宏才，李晓航

西南交通大学

计算机与人工智能学院

第1章 数据库系统概述

学习目的和要求



- 数据库管理系统出现的背景
- 数据库管理系统基本功能、抽象层次
- 数据库系统总体结构
- 理解数据库原理、应用及设计三部分间的关系
- 数据库系统中的术语与基本概念
- 数据库技术发展

1.1 数据库管理系统及其总体概述

从最原始的观点出发来看如下问题：

一.从利用文件系统来开发管理软件和网络共享观点来看待数据库管理系统的出现

1. 利用文件系统的应用软件开发过程

★ 开发任务及分析设计

简单学生管理系统，有学生注册、选课、学籍、和成绩等模块。

★ 开发工具及环境

C/C++、Windows操作系统的文件系统。

注：要完全抛开现成的数据库及工具。利用文件系统来模拟数据库。

★ 系统实现过程

⊙ 定义数据结构

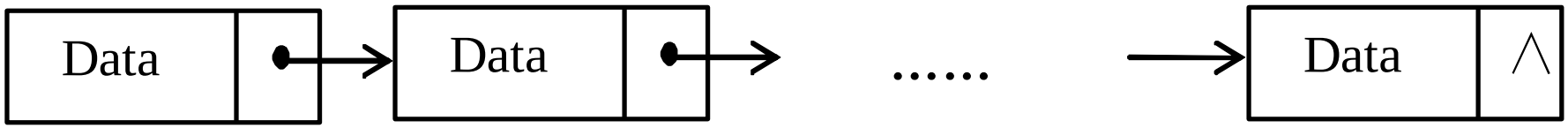
```
struct Student
{
        int     nStudNo;
        char    szStudName[20];
        char    cStudSex;
        int     nStudAge;
        char    szDept[30];
};
```

```
struct Enrolled
{
        int     nStudNo;
        int     nWhichTerm;
        char    cEnrolled;
        char    szMem[30];
};
```

```
struct Course
{
        int     nCourseNo;
        char    szCourseName[20];
        char    szDept[30];
};
```

```
struct Grade
{
        int     nStudNo;
        int     nCourseNo;
        int     nGrade;
};
```

⊙ 构造链表



注：链表中每个结点的数据（Data）段均为一个学生基本信息结构

学生基本信息链表

⊙ 操作链表

结点的插入（Insert）、删除（Delete）、修改（Update）和链表查询（Query）与**排序（Sort）**。

⊙设计用户界面（Graphical user interface, GUI）



学生学籍管理界面

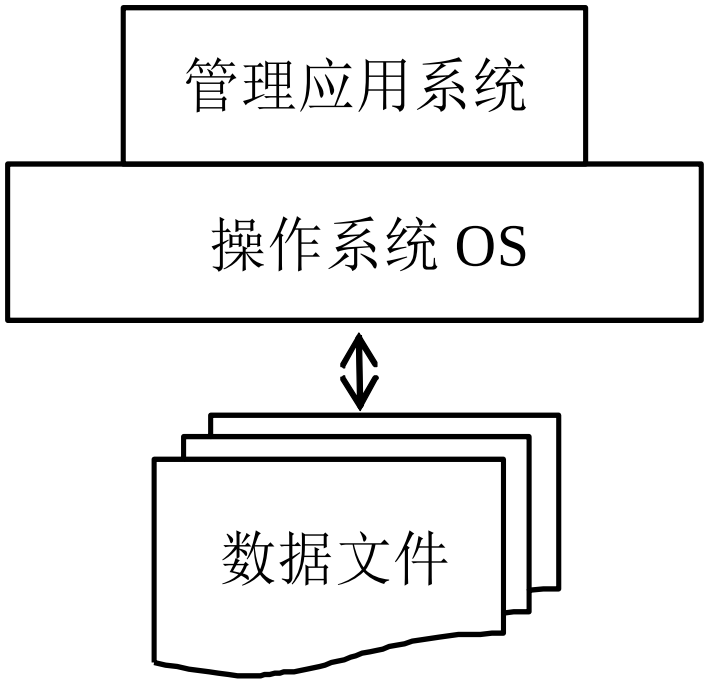
最基本的数据操作：增加、删除、修改和查询，简称：**增删改查**。业务操作或功能均以这四个最基本的数据操作为基础。

⊙ 创建数据存储文件

（*C语言里可采用 fopen、fclose 函数*）

⊙ 文件与链表的内外存数据交换

程序结束运行前，将链表中的数据存入到对应数据文件中；程序开始运行时，将文件中的数据读出到对应链表中（*C语言中可采用 fwrite 和 fread 函数*）。



基于文件系统的应用系统结构

2. 文件系统的缺陷

※ **大容量数据**存储，大数据量如500GB

1 KB(kilobyte)=1024 Bytes; 1 MB(megabyte)=1024 KB

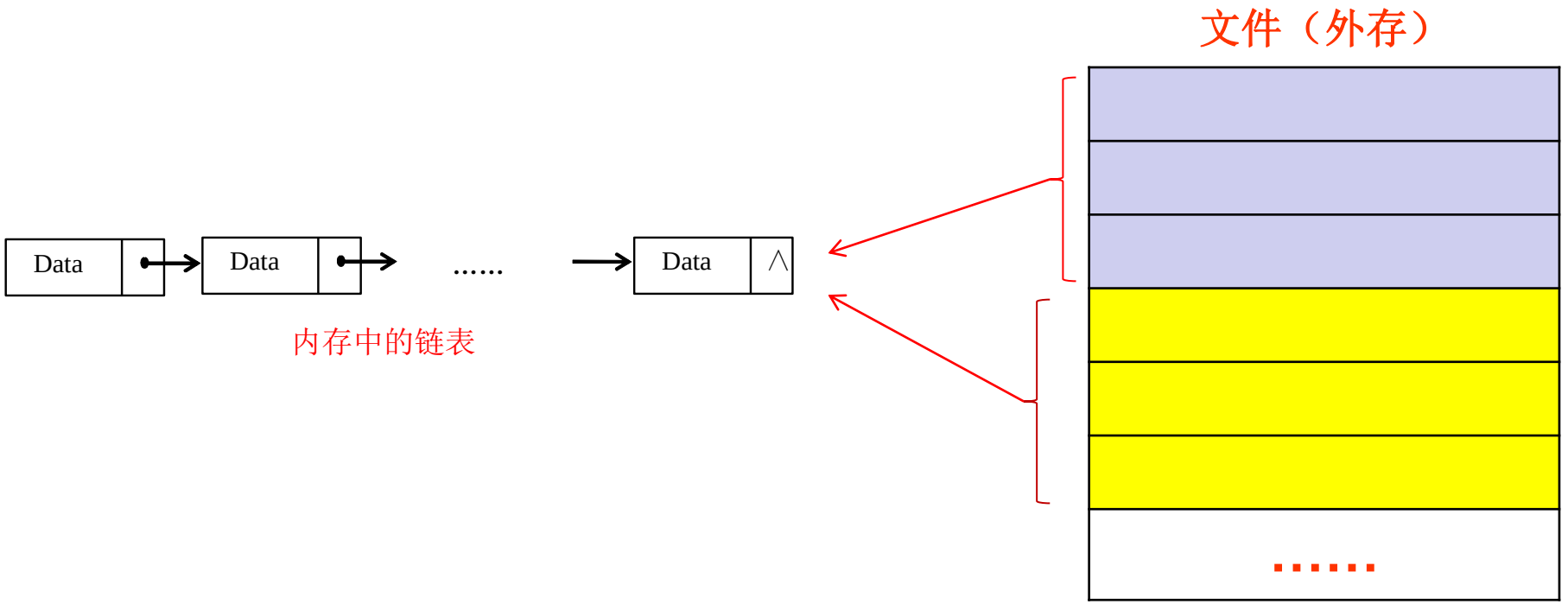
1 GB(gigabyte) =1024 MB; 1 TB (terabyte) =1024 GB

1 PB (petabyte) =1024 TB; 1 EB (exabyte) =1024 PB

1 ZB(zettabyte) =1024 EB; 1 YB (yottabyte) =1024 ZB

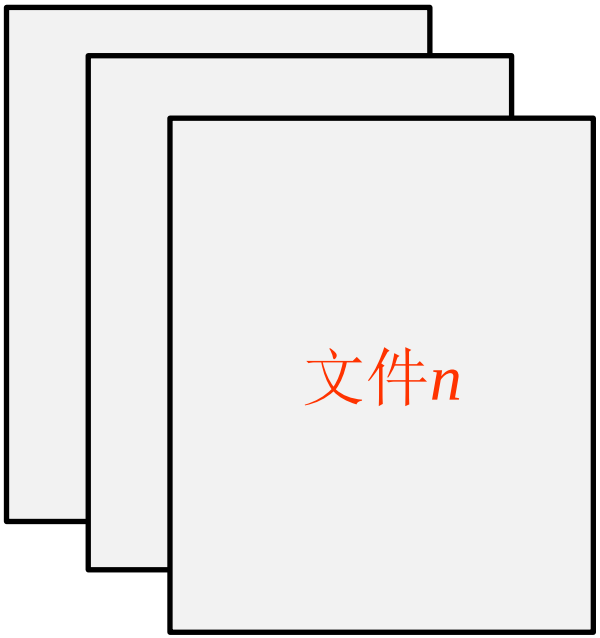
大数据量带来的问题：

（1）内存不够；



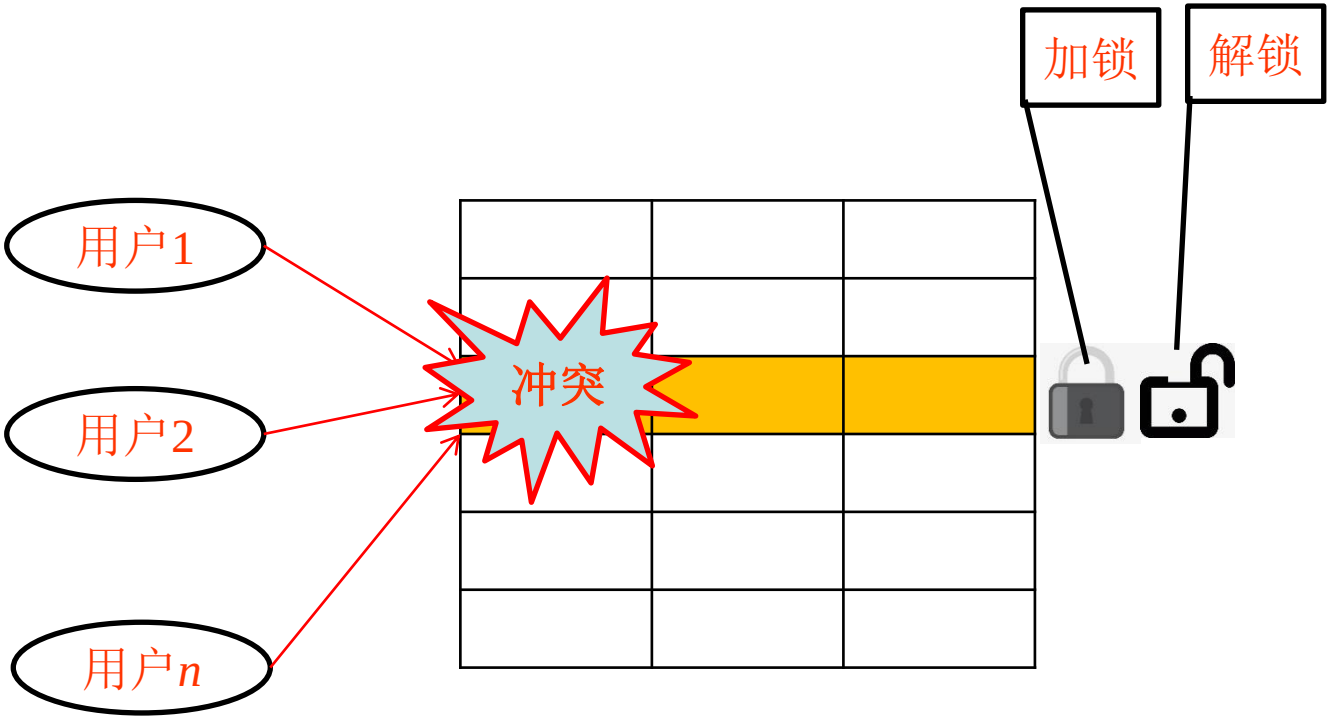
（2）32位计算机直接访问的地址为4GB；

在32位机上Linux、Windows等操作系统不允许硬盘上单个文件超过4GB（2^{32}寻址空间）大小。



（3）大数据量下的查询速度。

※ 多用户并发访问——并发控制



※ 数据完整性

如何保证存放于多处的数据一致性(某地的多处、本地与远程)

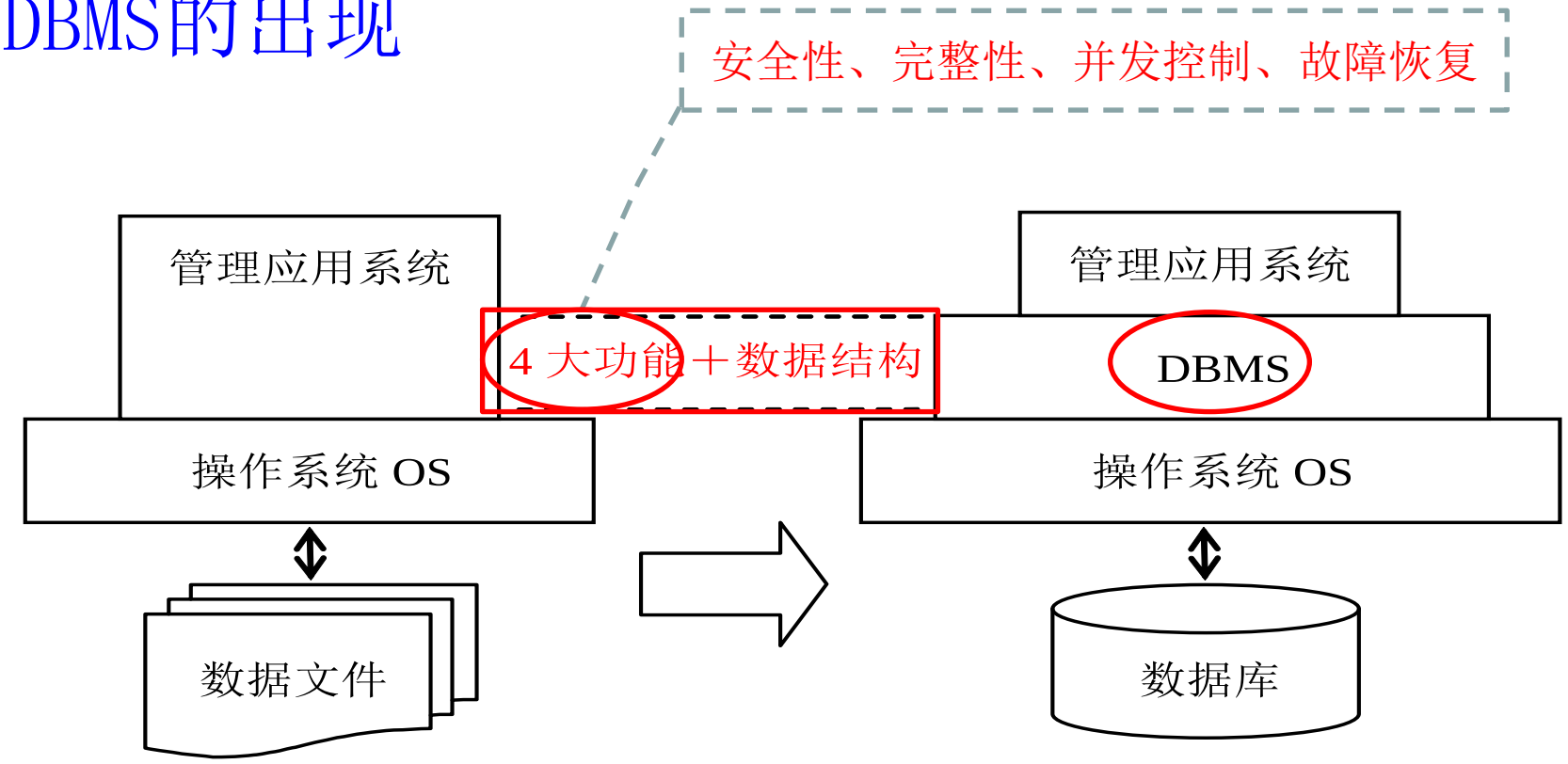
※ 故障恢复

系统出现故障（宕机、停电、闪崩、磁盘坏等）如何恢复数据。

※ 安全性

不同用户的授权问题等。

3. DBMS的出现



(a) 基于文件系统的应用系统结构　　(b) 基于 DBMS 的应用系统结构

应用系统体系结构变迁

4. 引起的术语变化及狭义对应

结构类型 → 数据模型

具体的某个结构 → 数据模式

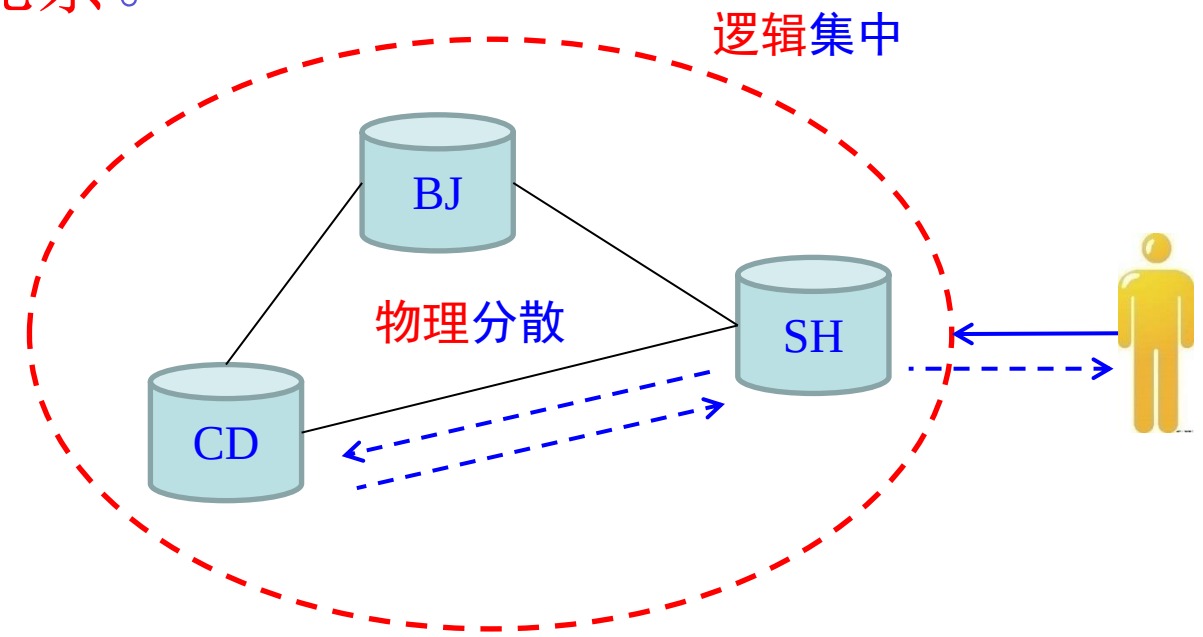
数据文件 → 数据库

二.从文件系统缺陷及管理特点来看数据库管理系统应具有的基本功能

通过将数据存储于DBMS中而不是文件系统中，可以以一种强壮、高效的方式进行数据的管理，其优点为:

- 数据独立性(Data Independence)：指应用程序独立于数据的表示（逻辑）与存储（物理），通过将数据的定义与存储从程序中独立出来实现。
- 高效数据访问(Efficient Data Access)：DBMS利用许多复杂的技术来高效存储和检索数据，这对存于外部存储设备上的数据相当重要。
- 数据完整性与安全性(Data Integrity and Security)：DBMS通过数据的完整性约束或限制(Integrity Constraints)、访问控制来完成。

◆ 数据管理(Data Administration)：数据的集中管理可减少冗余。



物理与逻辑的含意？

物理：外在的表现形式；逻辑：内在的处理方式。

软件：逻辑设计/物理实现

文件系统：逻辑文件/物理文件

数据库：物理分散/逻辑集中

网络：物理星形/逻辑总线、物理总线/逻辑环

◆ 并发访问与故障恢复(Concurrent Access and Crash Recovery)：并发访问可使用户感到好象只有他一个人在使用某个数据。

◆ 缩短应用开发时间(Reduced Application Development Time)：有许多重要的数据管理及其相关的任务由DBMS来完成，而非App.

※注：不使用RDBMS的环境

（1）苛刻的实时(Real Time)环境，如：前端控制；

（2）操作少，代码要求精炼；

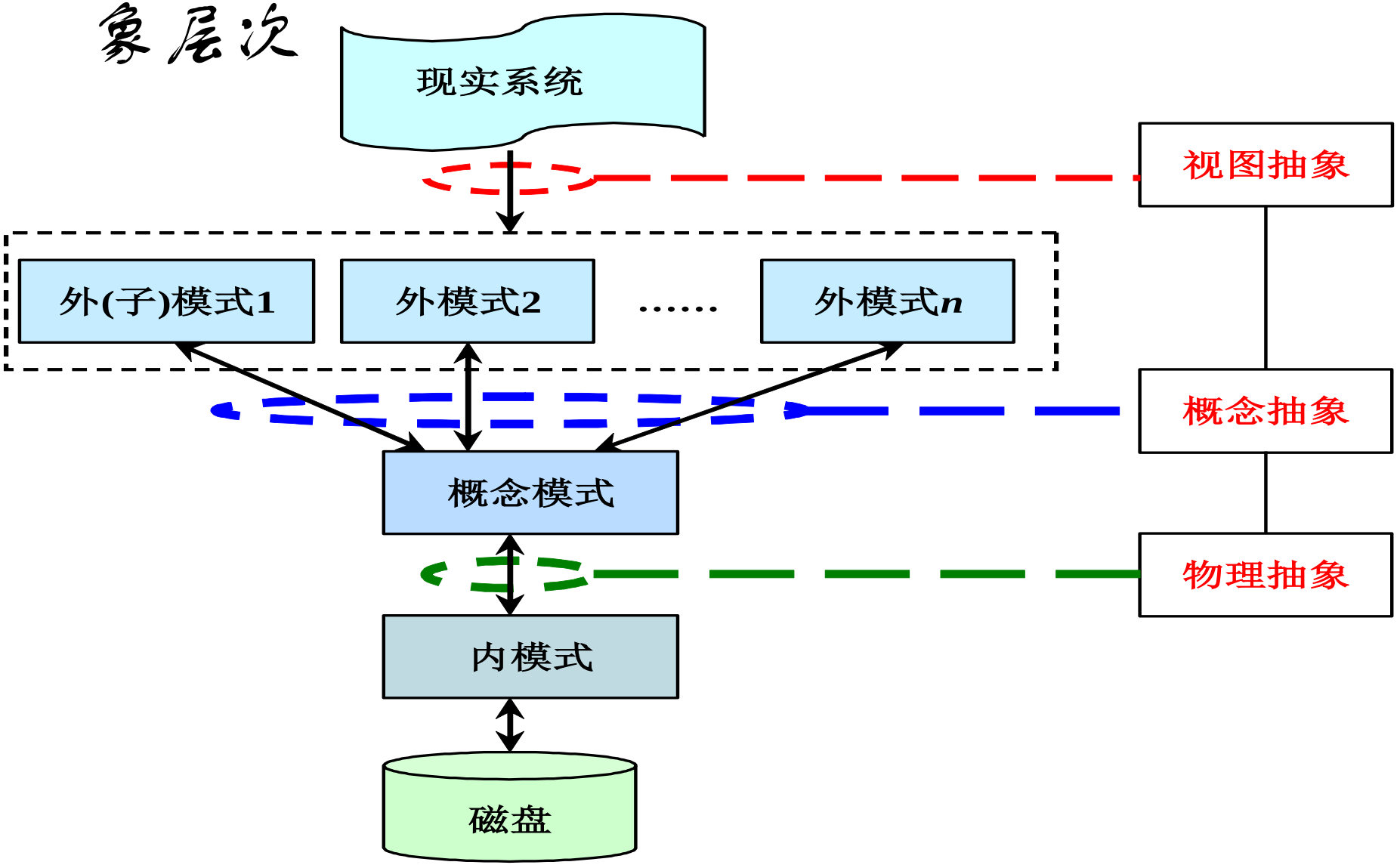
（3）操纵的数据（半结构化、非结构化）不被SQL语言支持，如文档数据、图像、音频/视频。

数据库管理系统应具有的基本功能

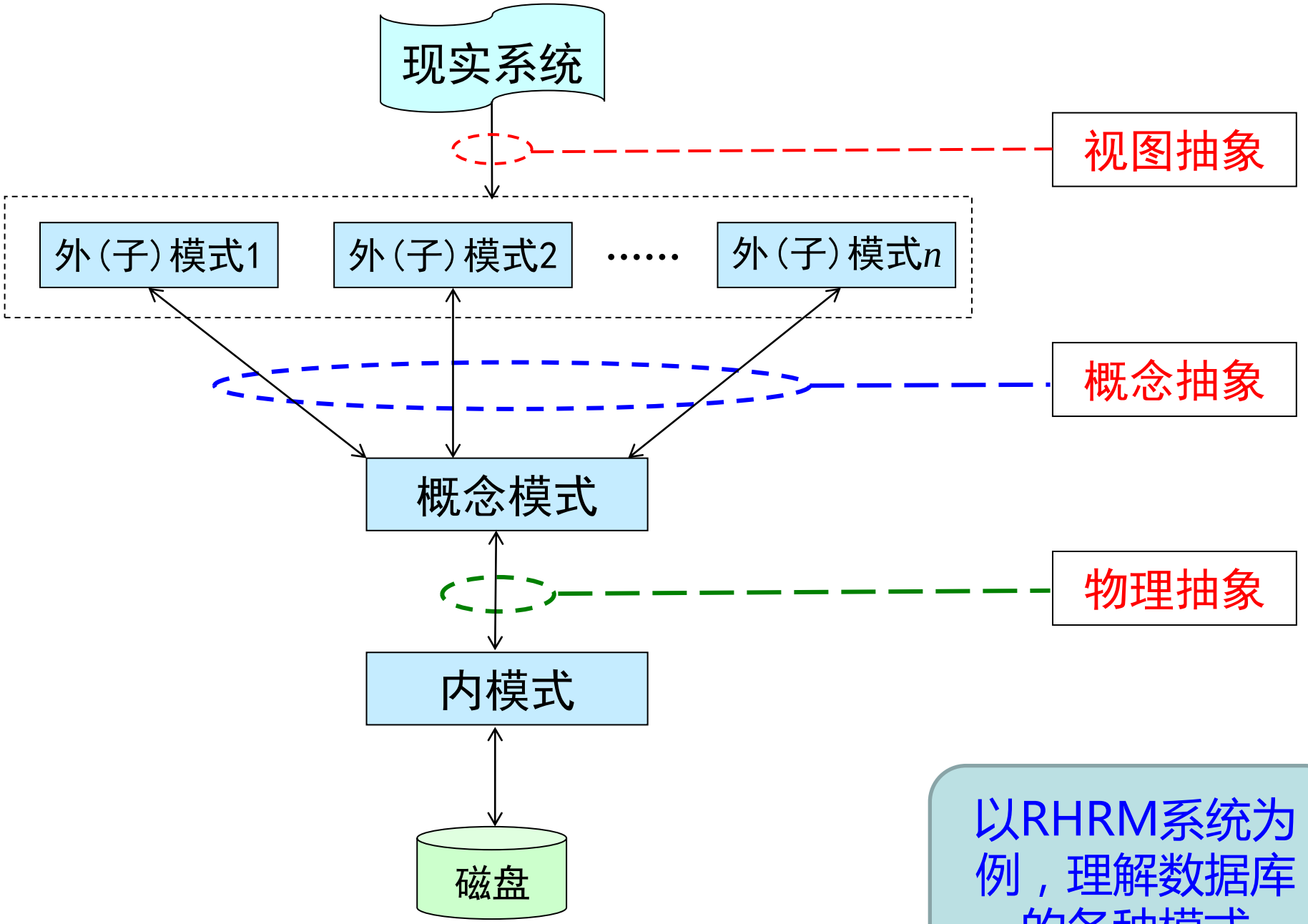
- 数据独立性
- 并发控制
- 故障恢复
- 安全性
- 完整性



三.从应用系统开发的角度来看待数据库的抽象层次

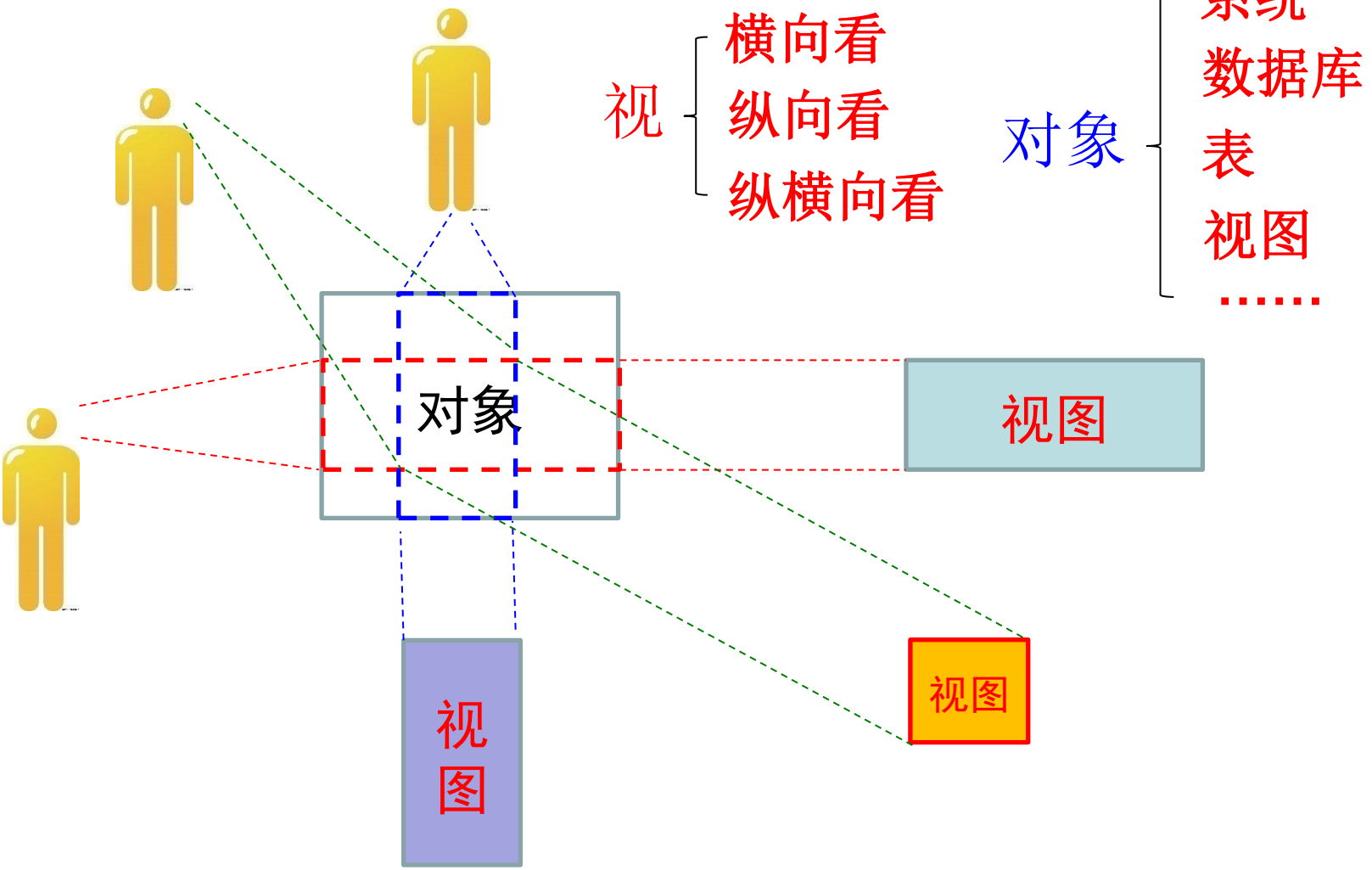


数据库系统抽象层次


现实系统
视图抽象
外(子)模式1
外(子)模式2
……
外(子)模式n
概念抽象
概念模式
物理抽象
内模式
磁盘
以RHRM系统为例，理解数据库的各种模式

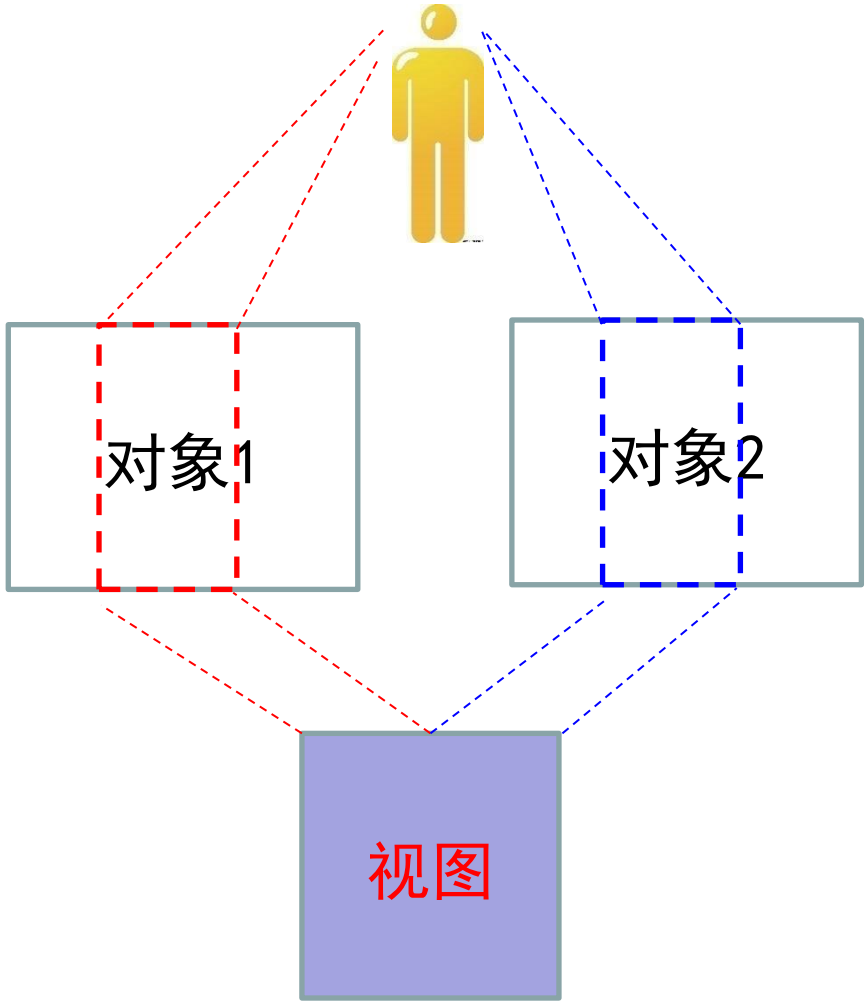
如何理解视图？

视图的原始定义：一个人“看（视）”一个对象所形成的图像。

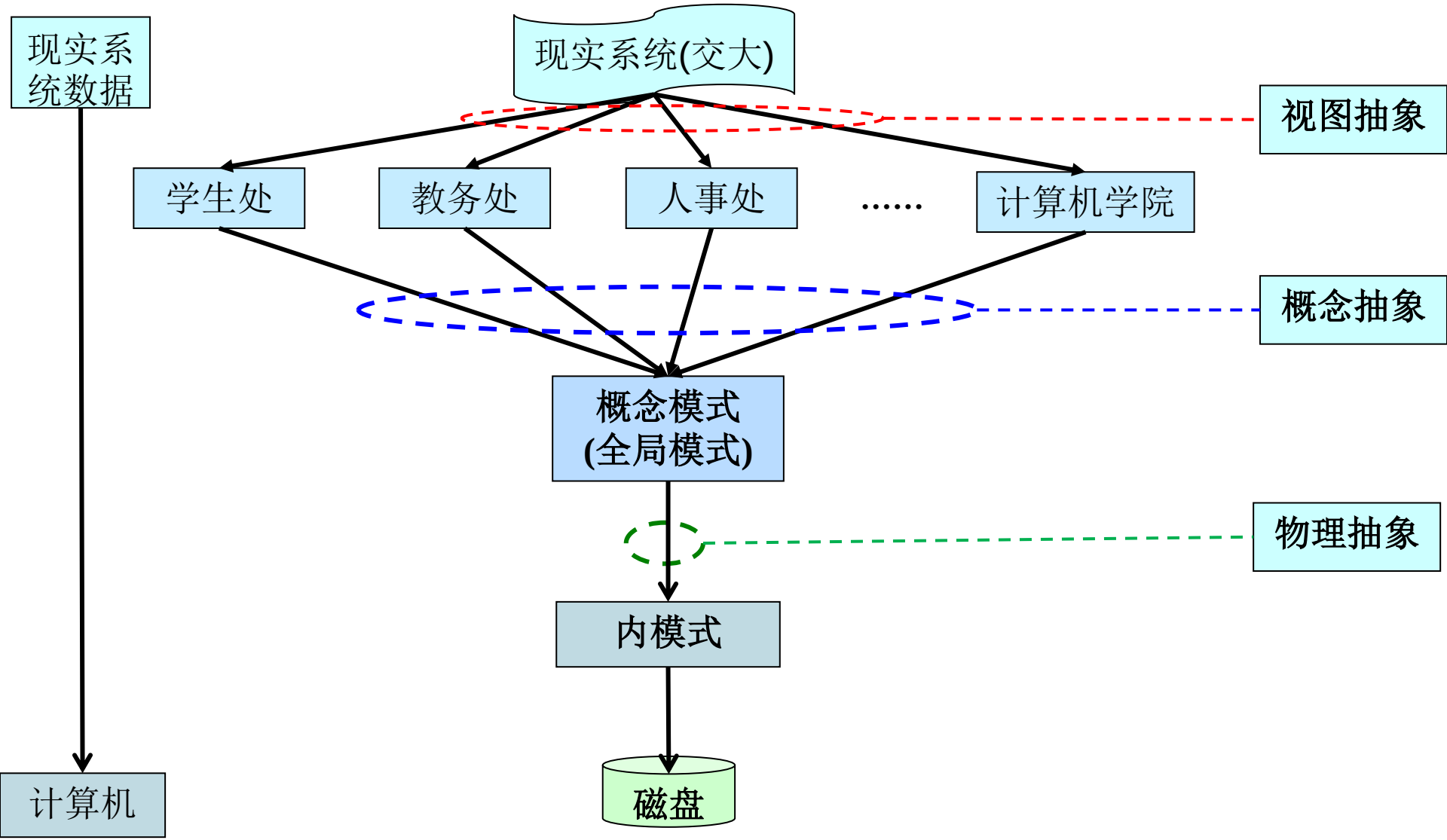


对视图原始定义的**扩展**之一

将一个对象扩展为多个同类对象。



举例进一步理解抽象层次



四. 从数据库的抽象层次来理解数据库的设计及其设计工具

- 抽象层次体现的正是数据库设计的过程与步骤
- 各过程阶段中使用不同的设计工具

视图抽象与概念抽象阶段——**高级**数据模型（如：ERM），也称概念数据模型或语义数据模型。

物理抽象阶段——**低级**数据模型（如：关系/层次/网状模型——**3个**传统数据模型），也叫逻辑数据模型。

数据模型分类
- 概念数据模型、语义数据模型、高级数据模型：ERM
- 逻辑数据模型、低级数据模型、传统数据模型：关系/层次/网状模型
- 物理数据模型

五．从对数据库管理系统的要求和操作来看待SQL数据库语言的功能

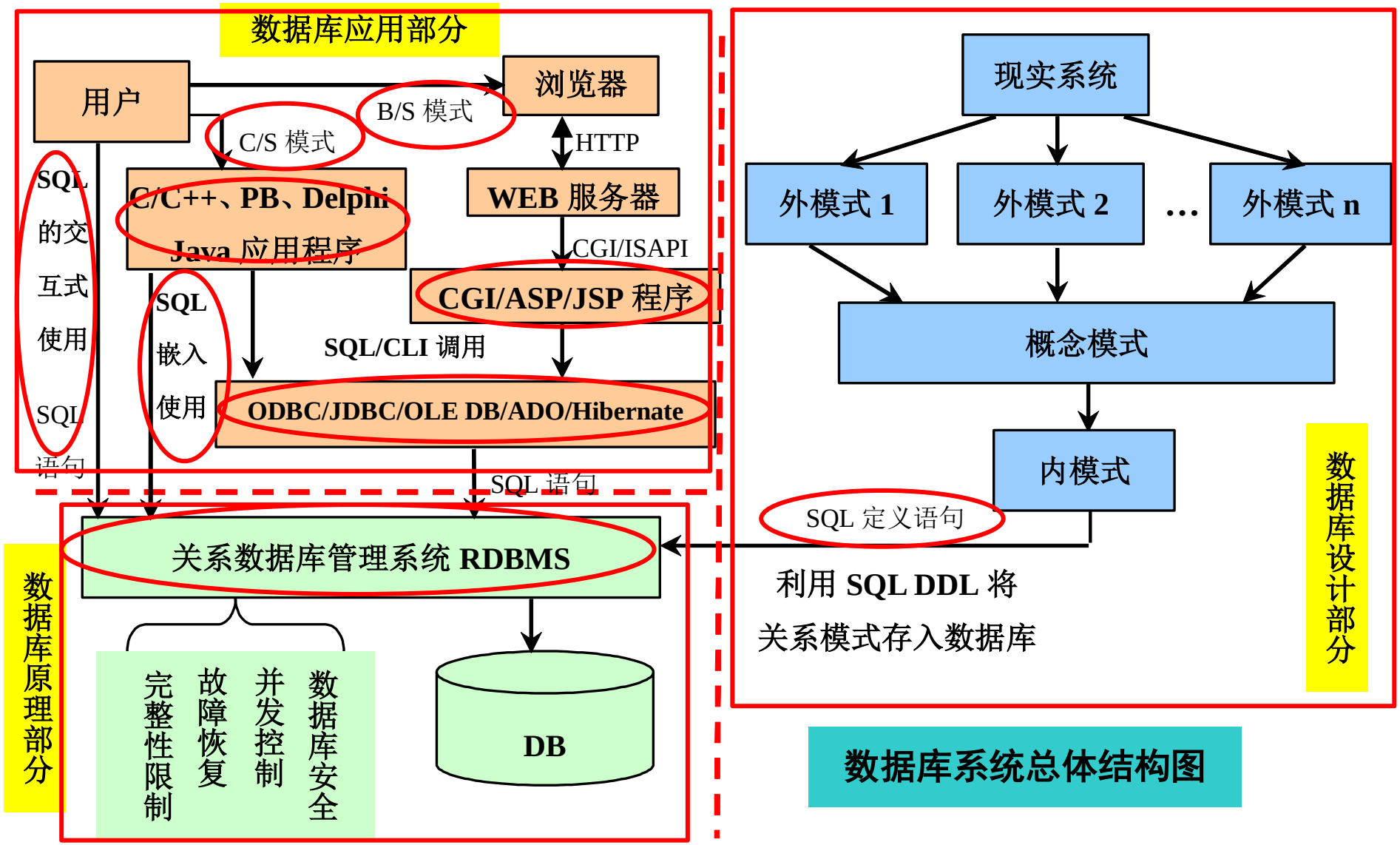
- 数据定义（Data Definition Language, DDL）
  语句：Create/Drop/Alter

- 数据操纵（Data Manipulation Language, DML）
  语句：Insert/Delete/Update/Select——增删改查

- 数据控制（Data Control Language, DCL）
  语句：较杂，含事务控制、安全权限控制等

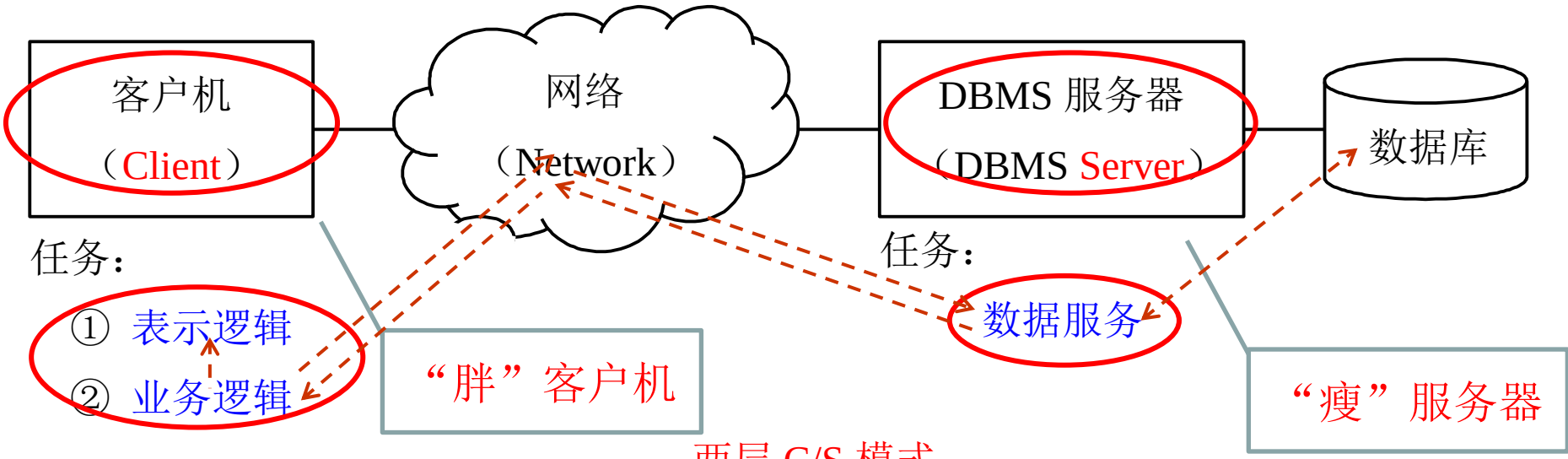
六. 数据库原理、应用与设计之间的联系

通过以上介绍应了解：数据库管理系统的原理与数据库应用系统的设计与开发的联系是紧密相关的。



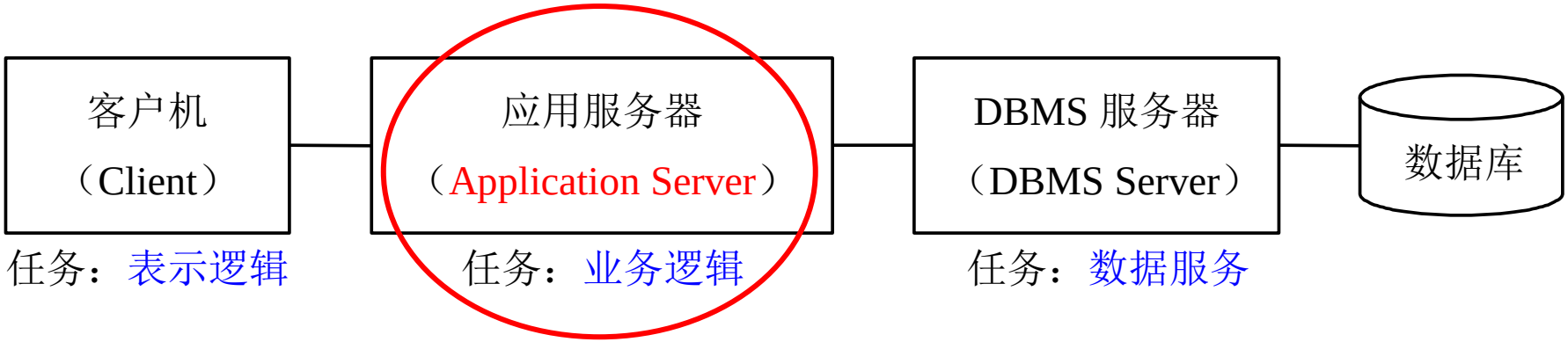
数据库系统总体结构图

C/S模式



两层 C/S 模式

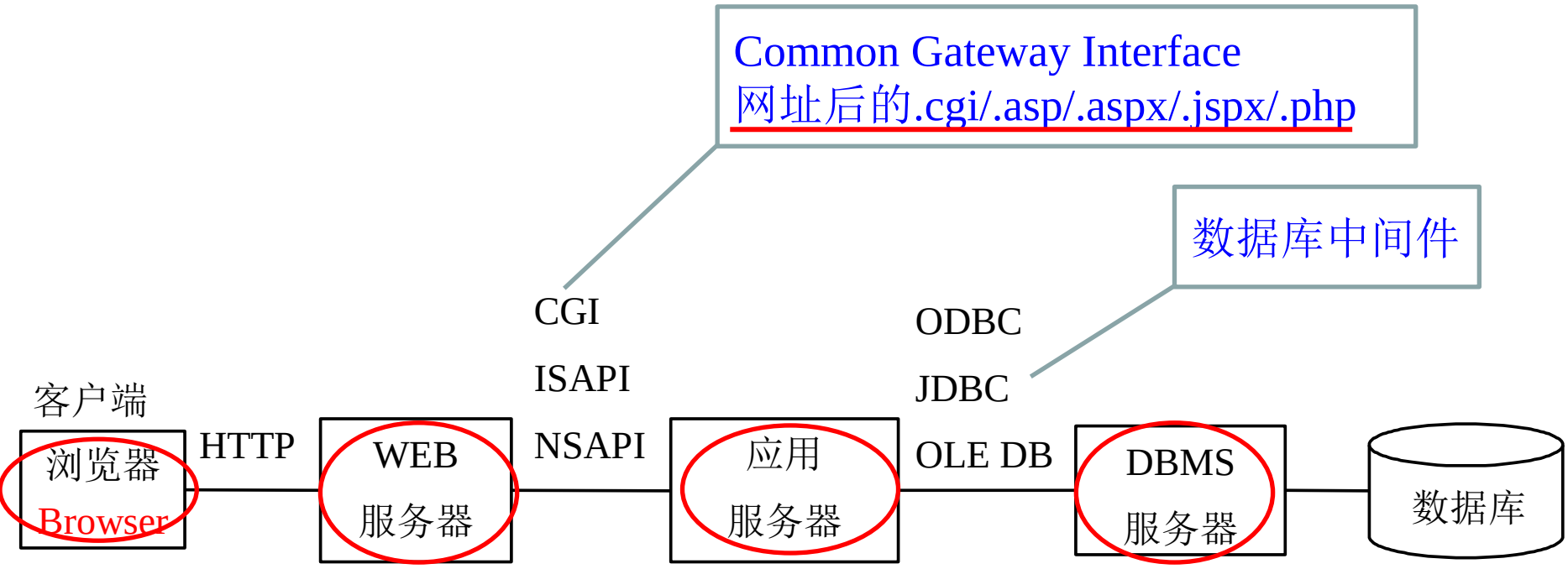
Two-tier Client/Server model



三层 C/S 模式

Three-tier C/S model

B/S模式



多层 B/S 模式结构

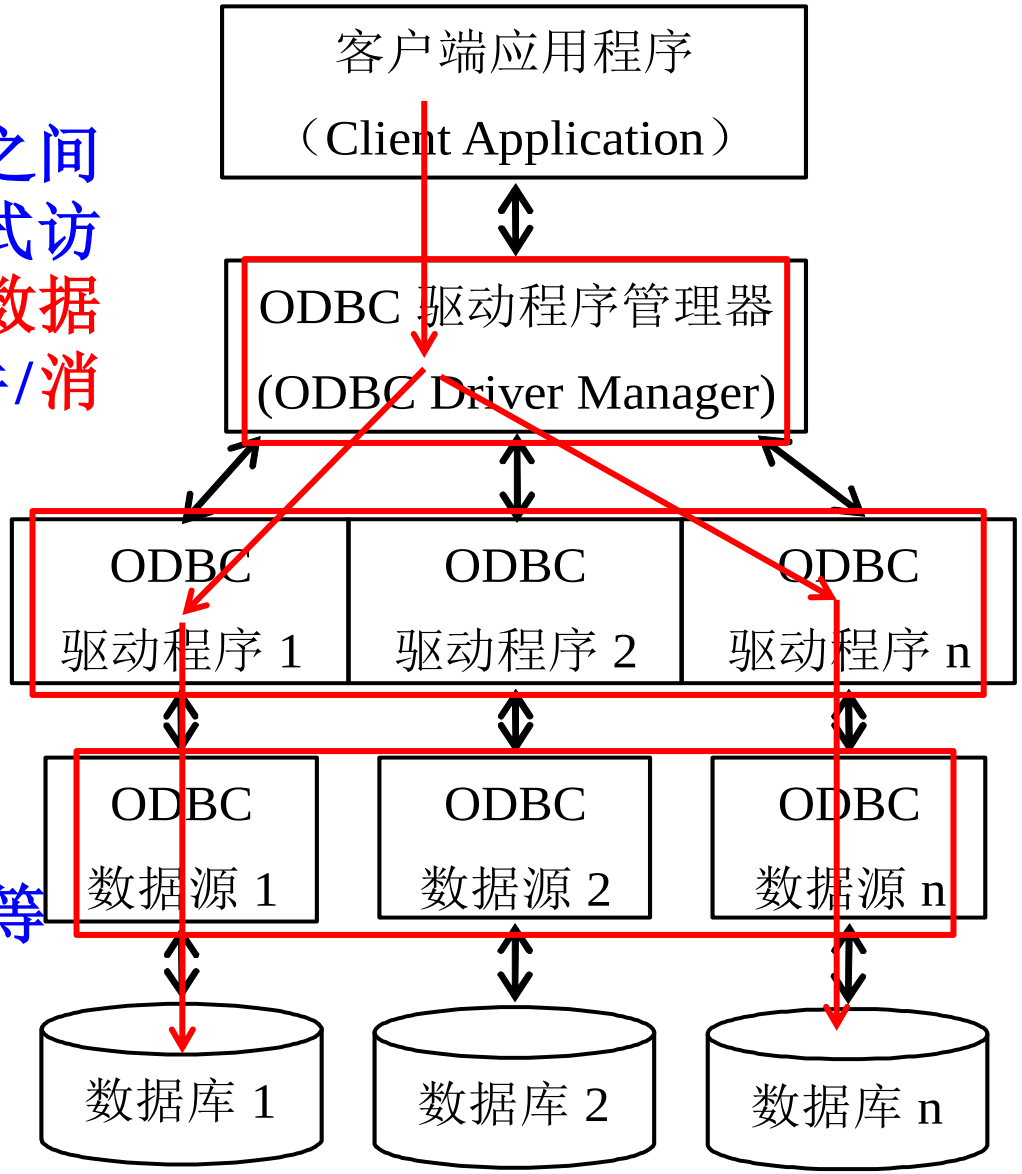
Multi-tier Browser/Server model

ODBC（Open Database Connectivity）开放数据库连接

中间件：通过屏蔽不同产品之间的差异，以统一的方式访问/连接不同的产品，数据库中间件/通信中间件/消息中间件……。

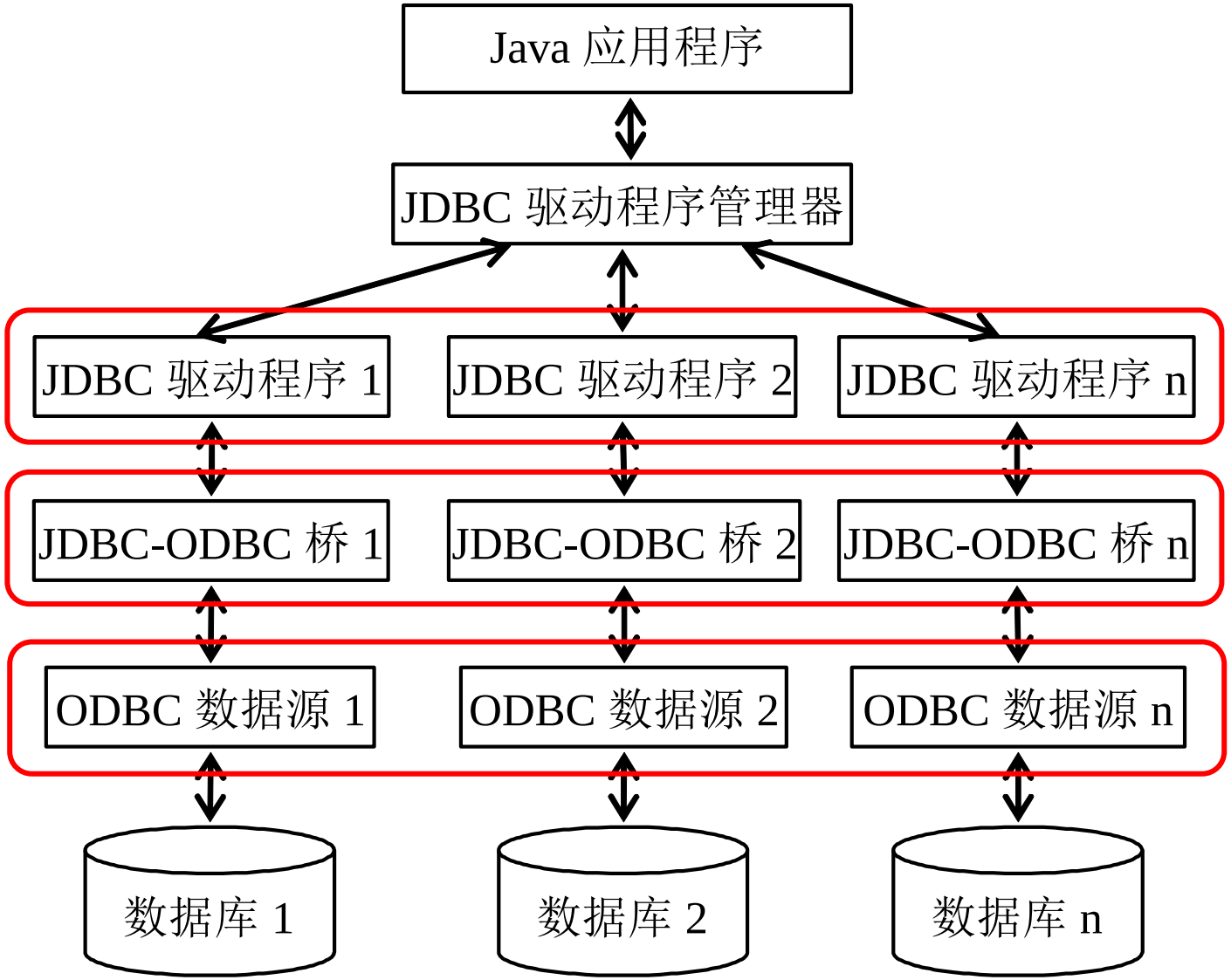
SQL 标准： SQL-86/89/92, SQL:1999/2003/2006/2008/2011/2016/2023

标准类别
- 强制标准：食品/卫生等
- 推荐标准：SQL等



ODBC 体系结构

JDBC（Java Database Connectivity）

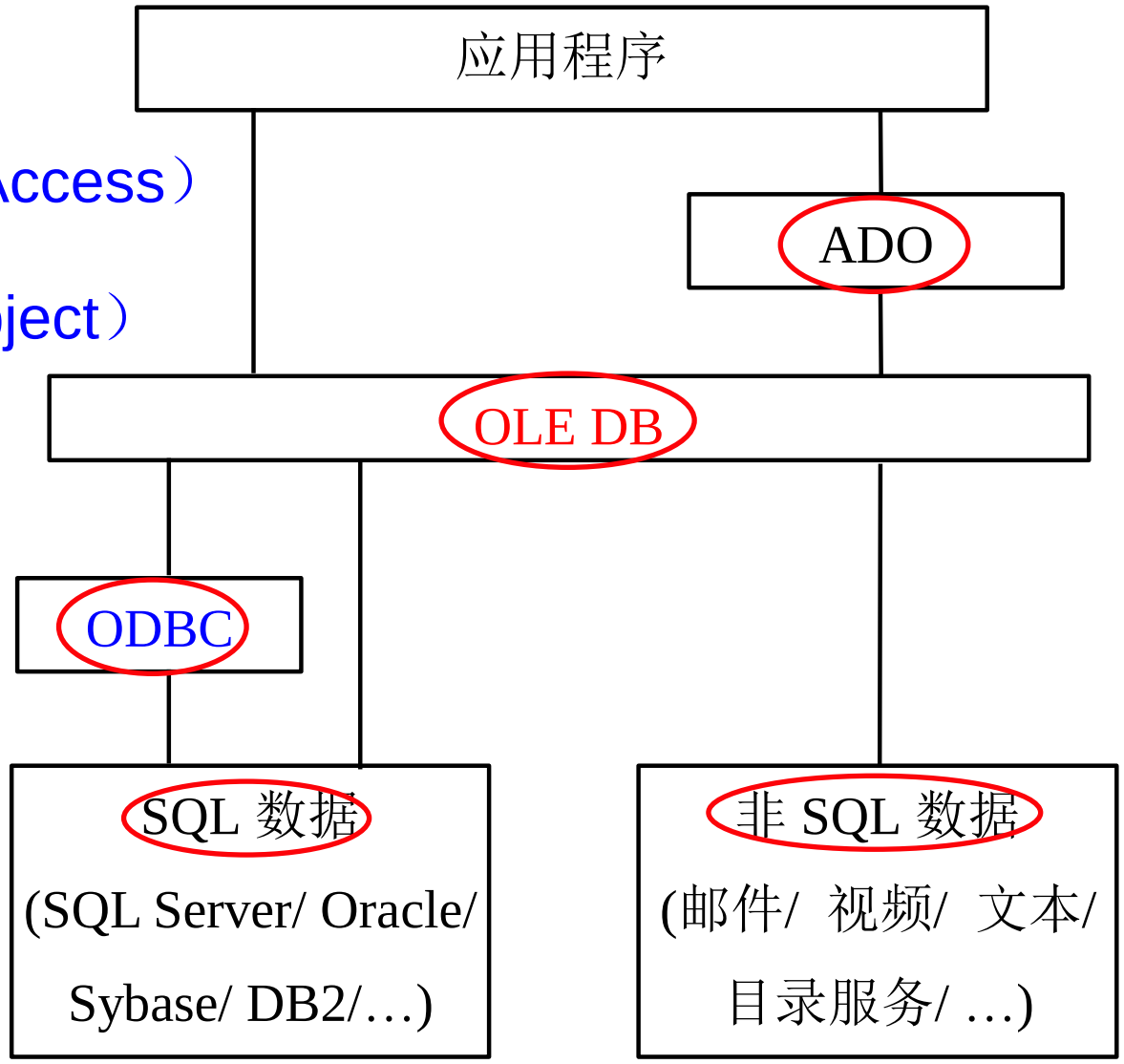


通过 JDBC-ODBC 桥访问数据库

OLE DB（Object Linking and Embedding Database）

UDA（Universal Data Access）

ADO（ActiveX Data Object）



UDA 层次结构

七. 从实际应用需要来看待数据库技术的发展

7.1 计算机的发明

- 数值计算
- 数据管理

7.2 数据管理走过的历程

1. 人工管理

(1) 所处时期：20世纪50年代中期—后期；

(2) 计算机状况

- 硬件(Hardware)：CPU、磁带(外存)；
- 软件(Software)：操作系统(Operating system, OS)功能有限。

(3) 特点：人工(程序员)参与程度高。

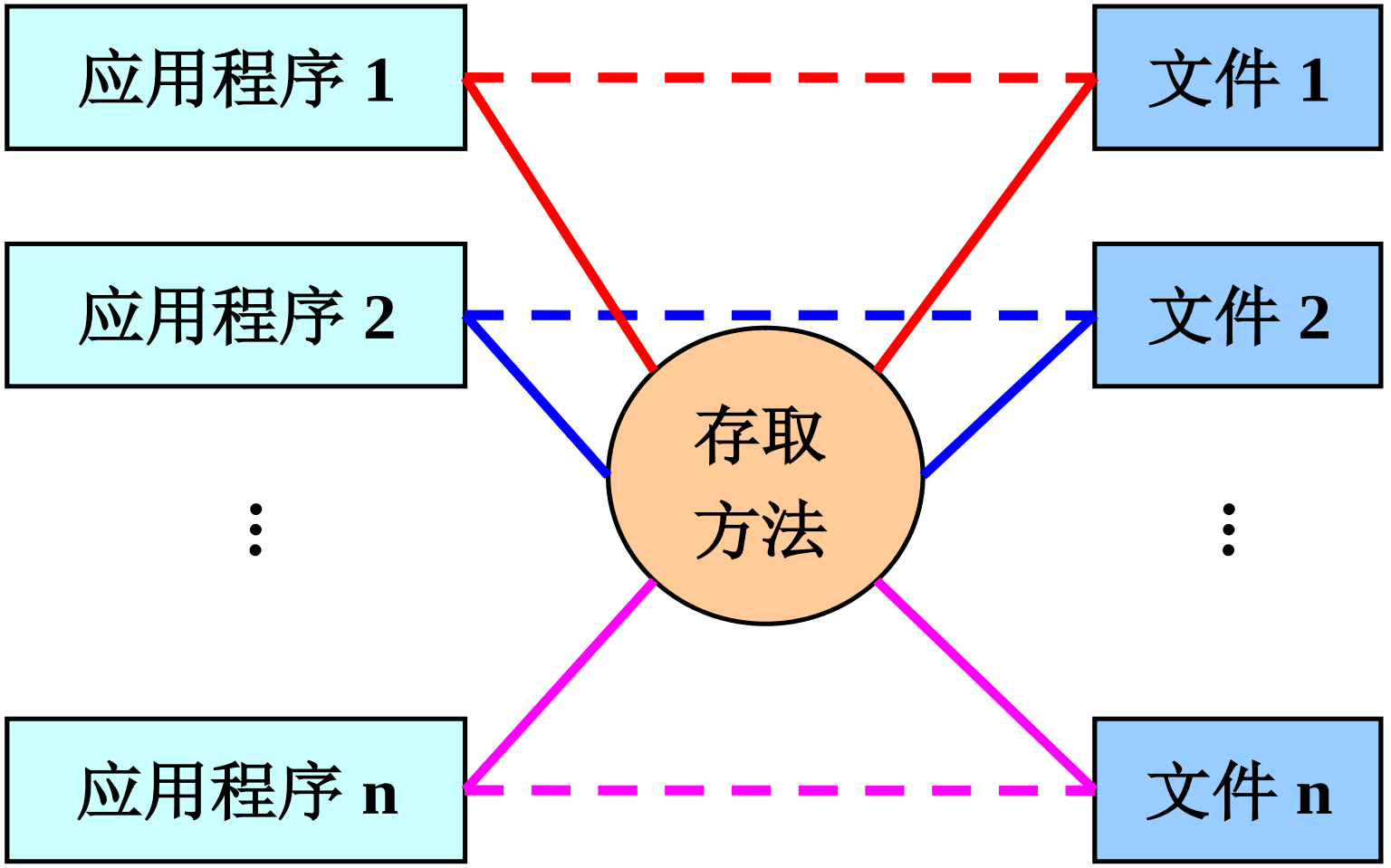
2. 基于文件系统(File system, FS)的数据管理

(1) 所处时期：20世纪50年代后期—60年代中期；

(2) 计算机状况

- 硬件： 磁鼓、 磁盘(Disk)→可直接存取(Access, or Read/Write, R/W)；
- 软件：OS提供文件系统管理，**逻辑**文件(抽象)⬌**物理**文件(具体)；

(3) 基于文件系统数据管理的结构


应用程序 1
应用程序 2
应用程序 n
存取
方法
文件 1
文件 2
文件 n

3. 基于数据库(Database, DB)的数据管理

(1) 所处时期：20世纪60年代—;

(2) 第2阶段的局限

- 数据独立性;
- 大容量数据及共享;
- 多用户访问;
- 故障恢复;
- 数据的安全性;
- 数据的完整一致性。

(3) 数据库管理系统发展的三代

① 第一代

✓ 时期：20世纪60年代。

✓ 层次数据库管理系统(Hierarchical database management system, HDBMS)。

代表：IBM的IMS(Information management system): 1968。

✓ 网状数据库管理系统(Net DBMS, NDBMS)。

代表：美国通用电气的IDS(Integrated data store): 1964。

✓ 典型人物：Charles William Bachman

——数据库界的第一位图灵奖得主(1973)。

——成就：IDS、DBTG(DB task group)报告。

——网状数据库之父。





✓ 局限：

- 须熟悉数据库结构；
- 较高编程技巧；
- 所支持的层次数据模型和网状数据模型缺乏相应数学基础。

②第二代：关系数据库管理系统(Relational DBMS, RDBMS)

✓ 代表性产品/项目

IBM System R (项目)、Ingres 11.2、IBM DB2 12.1 (Informix v14)、Oracle Database 23c (MySQL 8.3)、SAP ASE (Sybase) 16、MS SQL Server 2022(https://www.microsoft.com/zh-cn/sql-server/sql-server-downloads/)

阿里巴巴（OceanBase）、**华为**（GaussDB）、**腾讯**（TDSQL）、北京平凯PingCAP（TiDB）、人大金仓(KingBase)、武汉达梦DM(Data Management)、南大通用Gbase(General base)、巨杉 (Sequoiadb)、上海跬智（Kyligence）、中兴通讯（GoldenDB）、……。

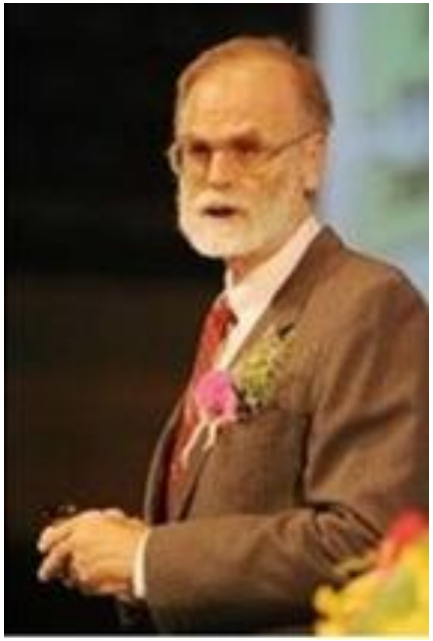


✓ 典型人物之一：Edgar Frank Codd

——数据库界的第二位图灵奖得主(1981)。

——成就：关系数据模型、关系数据库理论。

——关系数据库之父。



✓ 典型人物之二：James Gray(昵称Jim Gray)

——数据库界的第三位图灵奖得主(1998)。

——成就：共享数据库的一致性和锁的粒度、事务处理。

✓ 典型人物之三： Michael Stonebraker

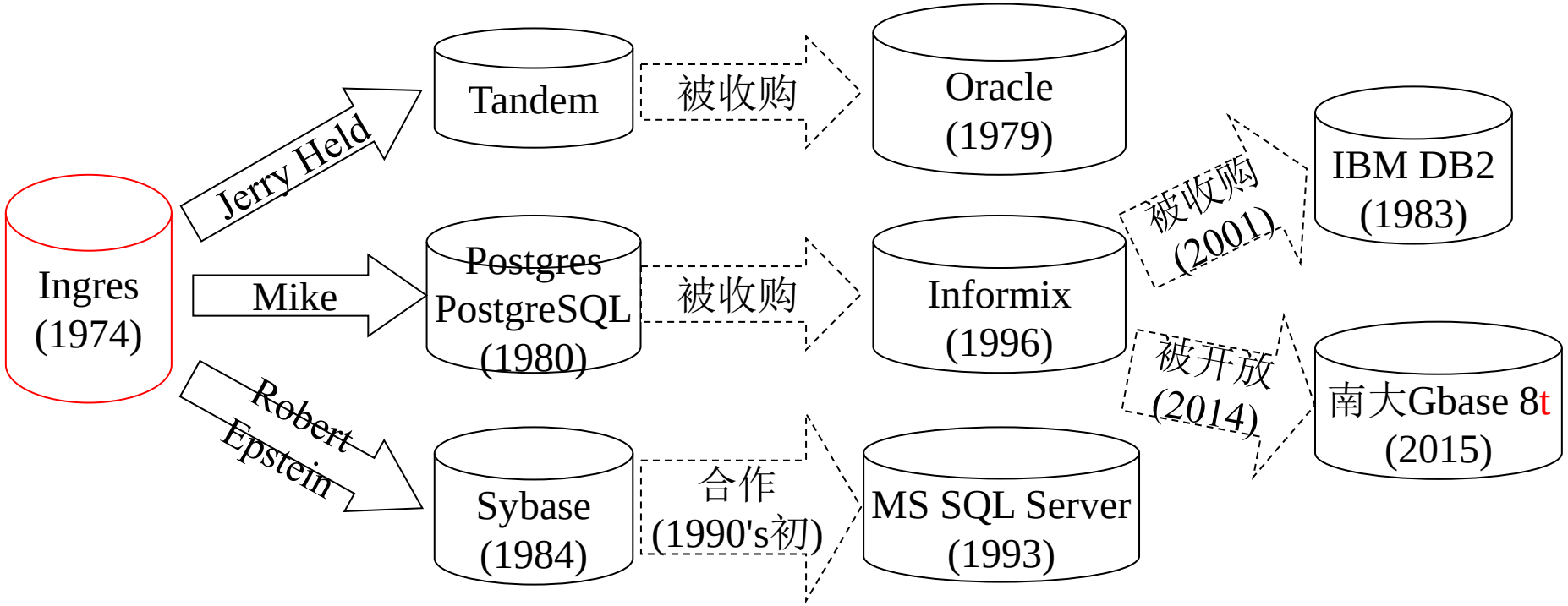
——数据库界的第四位图灵奖得主(2014)。

—— 成 就 ： Ingres(Interactive graphic and retrieval system)及其**开源**(Open source code)、开创性工作的商业化。

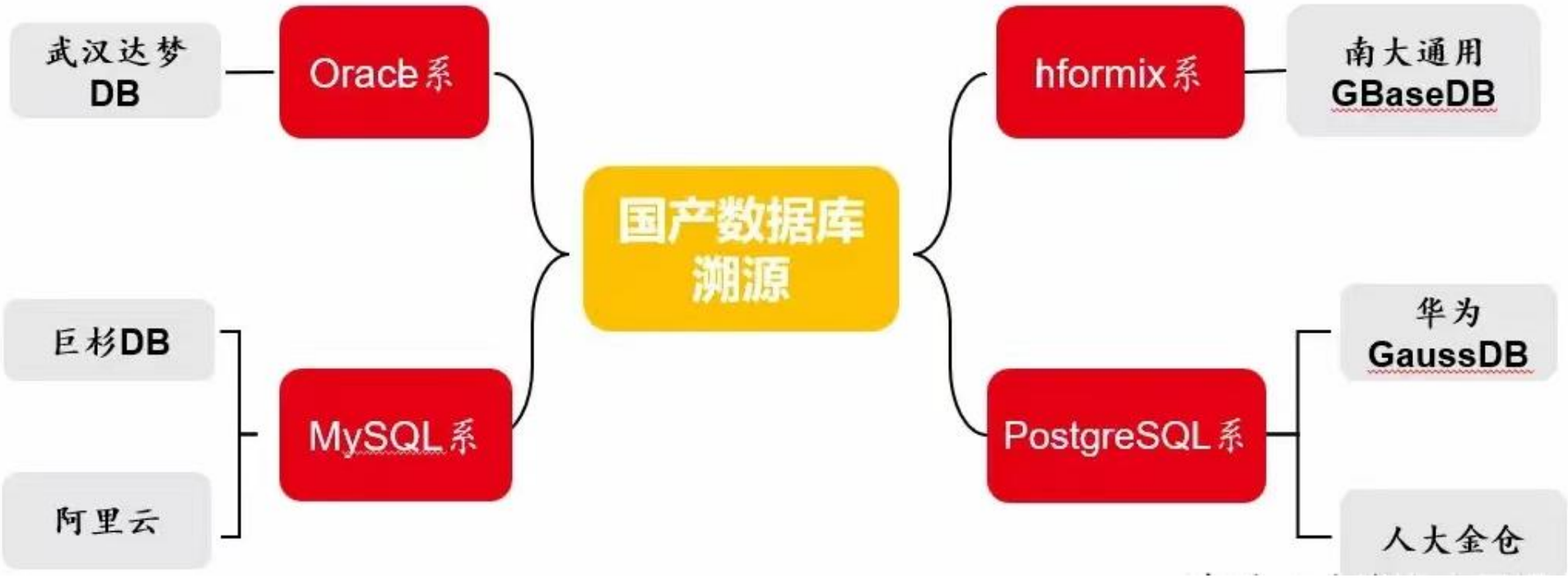




✓ Ingres的人才及开源带来的贡献



✓ 国产数据库技术流派



引自：鲁立“国产数据库发展之路”

✓ 2026年2月的数据库流行度排行榜 (DB-Engines)

429 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Oracle	Relational, Multi-model	1203.51	-33.82	-51.31
2.	2.	2.	MySQL	Relational, Multi-model	868.22	+0.70	-131.77
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	708.14	+1.88	-78.73
4.	4.	4.	PostgreSQL	Relational, Multi-model	672.03	+5.77	+12.42
5.	5.	5.	MongoDB	Document, Multi-model	378.73	+1.99	-17.90
6.	6.	↑ 7.	Snowflake	Relational	208.14	+0.34	+52.56
7.	7.	↓ 6.	Redis	Key-value, Multi-model	147.04	+2.88	-10.87
8.	8.	↑ 13.	Databricks	Multi-model	144.51	+2.97	+54.48
9.	9.	9.	IBM Db2	Relational, Multi-model	111.22	-1.50	-14.21
10.	10.	↓ 8.	Elasticsearch	Multi-model	106.46	-0.69	-28.17

Relational, Document store, Graph, RDF store

Relational, Document store

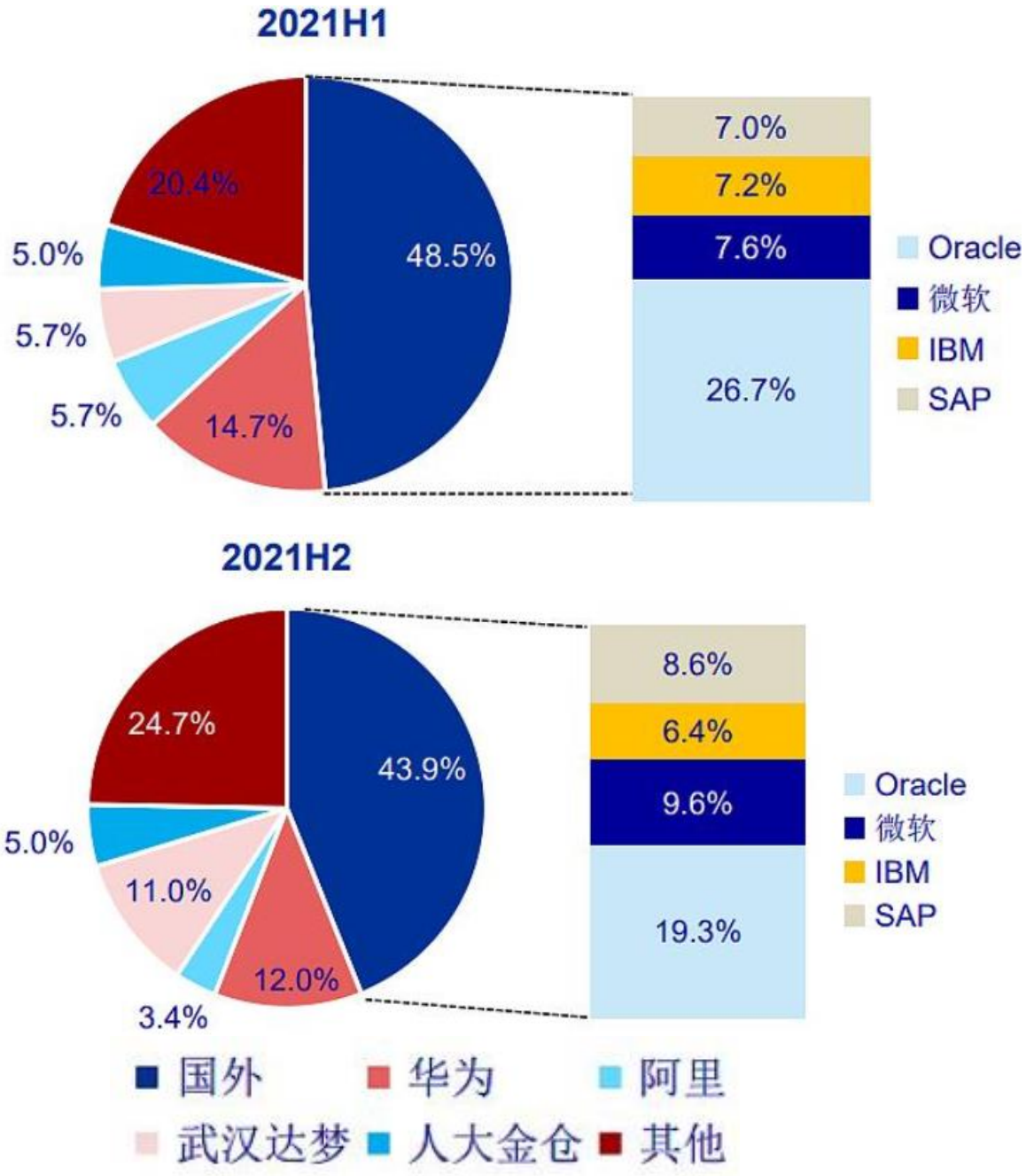
Relational, Document store, Graph

11.	11.	11.	Apache Cassandra	Wide column, Multi-model	101.60	+0.76	-0.98
12.	12.	↓ 10.	SQLite	Relational	99.19	-1.42	-14.63
13.	13.	↑ 14.	MariaDB	Relational, Multi-model	86.09	-1.64	-3.42
14.	↑ 15.	↑ 17.	Microsoft Azure SQL Database	Relational, Multi-model	73.66	-0.53	+0.90
15.	↓ 14.	15.	Splunk	Search engine	73.05	-2.25	-7.51
16.	16.	↑ 18.	Apache Hive	Relational	73.00	-0.48	+10.51
17.	17.	↓ 12.	Microsoft Access	Relational	69.47	-2.18	-27.07
18.	18.	↓ 16.	Amazon DynamoDB	Multi-model	67.87	-1.92	-7.71
19.	19.	19.	Google BigQuery	Relational	59.58	-4.02	+4.69
20.	20.	20.	Neo4j	Graph	49.26	-0.89	+3.92
21.	21.	↑ 24.	Apache Solr	Search engine, Multi-model	33.26	-0.84	+0.58
22.	22.	↑ 23.	Teradata	Relational, Multi-model	32.44	-1.25	-1.44
23.	23.	↓ 22.	SAP HANA	Relational, Multi-model	31.75	-0.72	-3.47
24.	24.	↓ 21.	FileMaker	Relational	30.19	-1.04	-10.03
25.	25.	25.	SAP Adaptive Server	Relational, Multi-model	26.18	-0.60	-2.44
26.	26.	↑ 27.	Microsoft Azure Cosmos DB	Multi-model	23.88	-0.92	+1.75
27.	27.	↑ 29.	PostGIS	Spatial, Multi-model	22.57	-0.29	+3.40
28.	↑ 30.	↑ 30.	ClickHouse	Relational, Multi-model	22.37	+0.60	+3.66
29.	↓ 28.	↑ 31.	Apache Spark (SQL)	Relational	21.90	-0.78	+3.45
30.	↓ 29.	↓ 28.	InfluxDB	Time Series, Multi-model	21.47	-0.45	-0.30

墨天轮中国数据库2026.2排行榜(https://www.modb.pro/dbRank)

排行	上月	半年前	名称	模型	数据处理	部署方式	商业模式	专利	论文	案例	资质	书籍	得分
1	↑ 2	↑↑↑ 4	PolarDB	关系型	TP HP			592	75	95	14	2	722.50
2	↓ 1	↓ 1	OceanBase	关系型	HP			151	53	57	16	3	710.35
3	3	↑↑ 5	达梦数据库	关系型	TP			518	11	91	14	18	679.78
4	↑ 5	↓ 3	金仓数据库	关系型	TP AP			841	1	117	13	6	604.72
5	↓ 4	↓↓↓ 2	GoldenDB	关系型	HP TP			905	64	115	12	2	558.37
6	↑↑ 8	↑↑ 8	TiDB	关系型	HP			40	29	19	10	2	490.18
7	↓ 6	7	TDSQL	关系型	HP			110	56	47	17	0	410.08
8	↓ 7	↓↓ 6	GBASE	关系型	AP TP			356	0	133	14	0	404.36
9	9	9	GaussDB	关系型	TP HP			950	65	36	17	5	362.57
10	10	↑ 11	YashanDB	关系型	HP			95	39	17	10	0	349.20

✓中国数据库市场份额占比



截止2025年上半年，国产数据库整体份额达**71%**，无论是公有云还是本地部署市场，国产数据库都已经占据了主导地位。

③第三代：有少量产品出现，但未形成主流

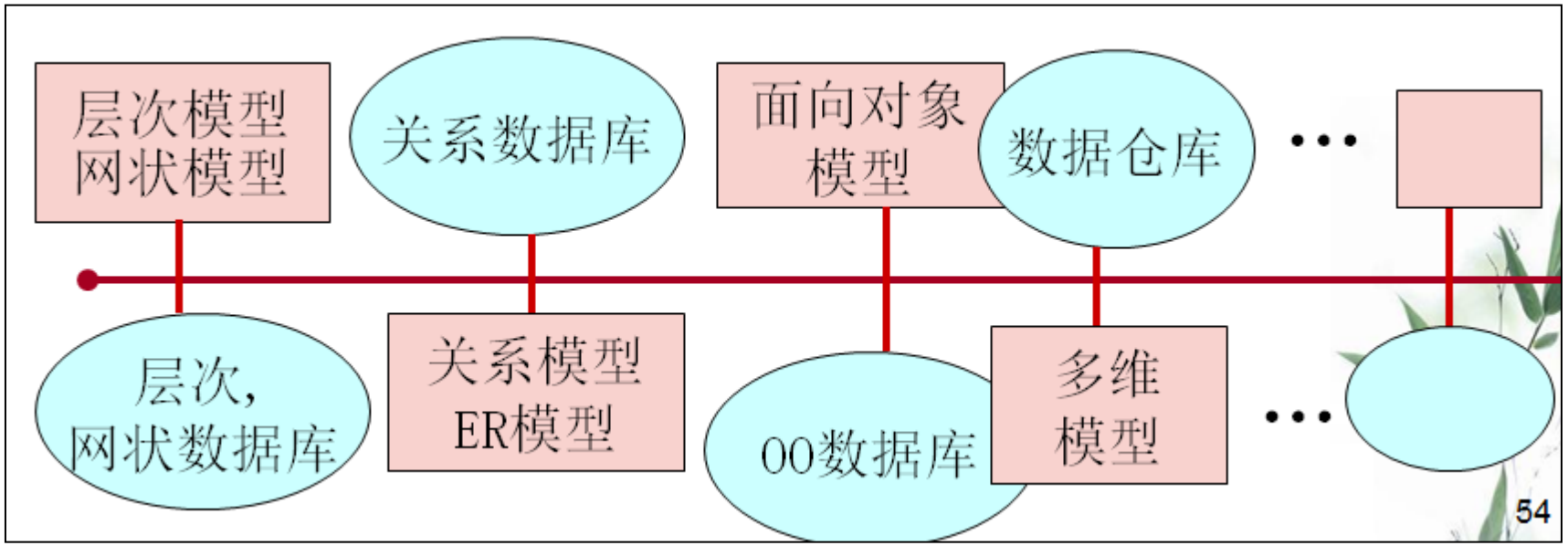
- ✓ 标志：面向对象数据库管理系统(Object-oriented DBMS, OODBMS)。
- ✓ 实现途径：

——ORDBMS(Object-relation DBMS)；

——面向对象程序语言+扩充的面向对象的数据模型；

——纯支持对象数据模型。

数据库系统划代小结



7.3 OLTP与OLAP

1. OLTP

——Online transaction processing, 联机事务处理，可等同MIS(Management information system, 管理信息系统)。

——指一类应用，如：火车票/飞机票订售、商场收银、银行储蓄，高校教务/学生/人事/财务管理，……。

(1) 特点

- 面向组织中低层；
- 完成日常事务处理；
- 响应速度迅速，≤5s。

(2) 目标

- 提高工作效率；
- 降低劳动强度。

2. OLAP

——Online analysis processing, 联机分析处理，等同DSS(Decision support system)。

——概念首次由E. F. Codd提出(1993年)。

——特点

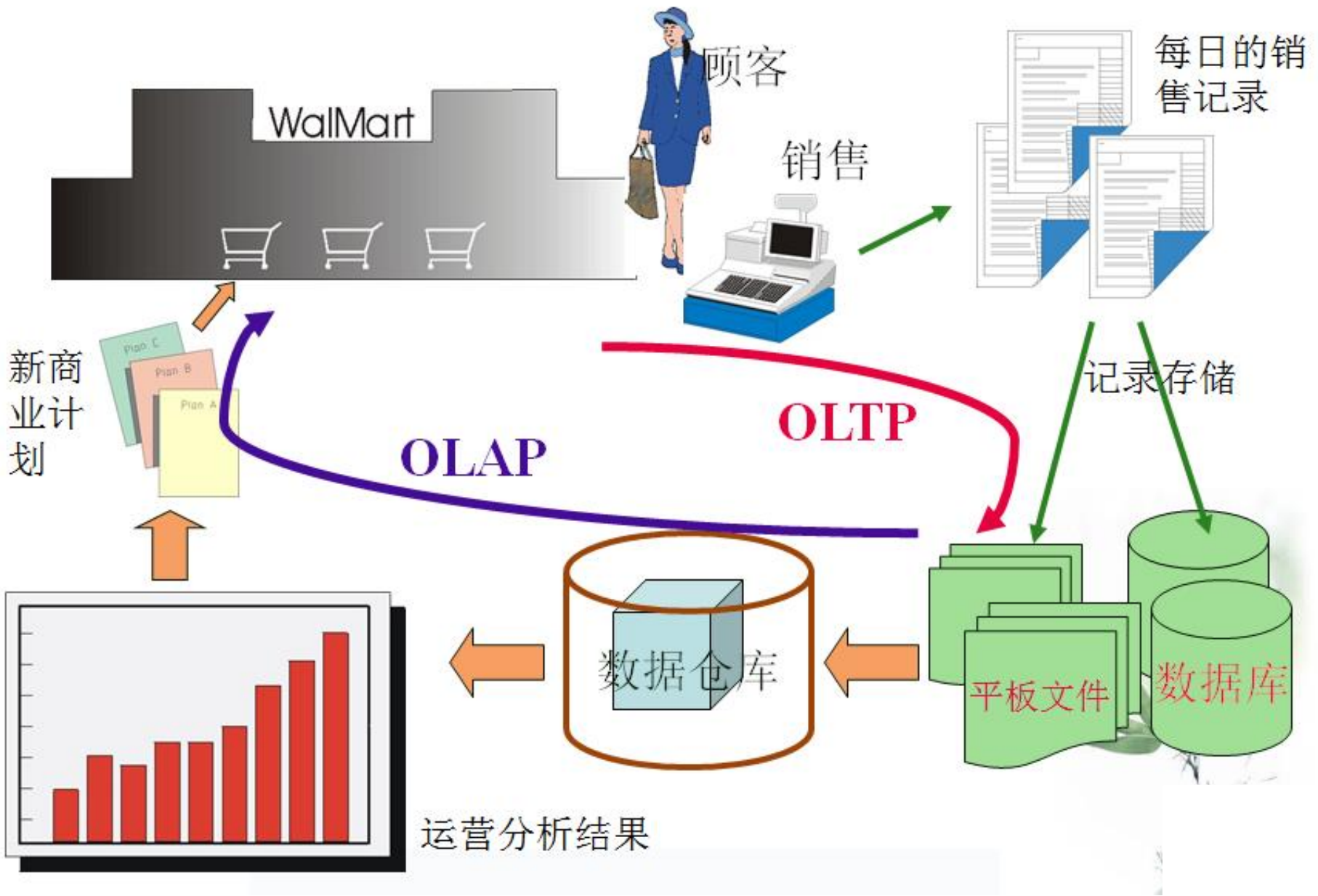
- 面向组织高层；
- 注重**多维**数据分析；
- 响应时间未做特别要求。

——目标：辅助决策与分析。

3. HTAP (Hybrid Transactional/Analytical Processing)

即混合事务/分析处理，其核心目标就是在同一个数据库系统中，同时高效地支持联机事务处理（OLTP）和联机分析处理（OLAP）。

OLTP and OLAP



7.4 数据仓库与数据挖掘

1. 数据仓库(Data warehouse, DW)

——经典事例：啤酒+尿布。

——概念由W. H. Inmon提出(1991年)。

——特点

- 面向主题，如产品、客户、供货商……；
- 集成：多维数据；
- 稳定：历史数据；
- 时序数据。

2. 数据挖掘(Data mining, DM)

——啤酒+尿布：尿布→啤酒，关联(日常较难想到的对象之间)模式，知识表达的一种。

——等同知识发现KDD(Knowledge discovery in database, 1989年提出)，DM则于1995年正式被提出。

(1) 主要任务：从海量数据中挖掘出隐藏的知识模式。

(2) 常用的知识模式：关联模式、分类模式、聚类模式、时序模式、……

(3) 常用技术：规则归纳、决策树、神经网络、遗传算法、模糊方法、粗集方法、……

(4) 涉及的学科领域

机器学习/深度学习、人工智能(Sora（OpenAI）/Gemini(Google)/XAgent(清华)/ChatGPT(OpenAI)/文心一言(百度)/讯飞星火/AlphaGO)、模式识别、归纳推理、统计学、数据库、并行计算/云计算、大数据、……

(5) 应用的行业

商业零售、金融保险、医疗卫生、电信/移动、司法/刑事、工业生产、道路交通、选民拉票、推荐服务、……

7.5 各类应用的数据库系统

(1) 对象-关系数据库系统

——在关系模型中引入对象特征(如继承性)。

(2) 并行数据库系统

——将并行处理(多CPU、多核CPU)技术引入数据库系统。

(3) 地理空间数据库系统

——地理信息系统(Geographic information system, GIS)。

(4) 分布式数据库系统-云数据库

——将分布式(Distributed)技术引入数据库系统。

(5) 其它数据库系统

- 嵌入式移动数据库系统
- 工程数据库：CAD/CAM
- 主动(Active)数据库
- 向量数据库(VectorDB)

7.6 主/内存数据库(MMDB/IMDB)

1. 出现背景

(1) 应用需求

- ✓ 数据库的响应速度快；
- ✓ 在线事务吞吐率高。

(2) 芯片集成度提高，内存价格急剧下降

2. 概念

MMDB（Main memory database）或IMDB（In-memory database）把数据全部或当前工作部分驻留在内存中，应用程序与数据库的交互只需访问内存即可，从而消除了I/O瓶颈，获得了**CPU直接访问数据库数据**的极高存取速度，极大地提高了系统性能和吞吐率。

3. 产品

(1) 开源产品

SQLite、 MonetDB、 H2、 HSQLDB、 Derby、 FastDB、 Oracle Berkeley DB等。

(2) 商用产品

Oracle TimesTen、 IBM SolidDB、 SAP HANA、 Altibase、 eXtremeDB、Sybase ASE(-IMDB)、MS SQL Server的Hekaton等 。

7.7 云数据库

(1) 提供的主要服务：云数据库服务

云数据库：Google的Bigtable、Amazon的Aurora、Facebook的Cassandra、Yahoo的Sherpa/PNUTS、Oracle Database 12c、Microsoft的Azure、阿里的PolarDB、腾讯的TencentDB、华为的GaussDB。

(2) 主要挑战

- ✓ 数据的物理位置——由供应商控制而非用户，可能影响性能（查询优化、远程访问）；
- ✓ 复制问题——需保证访问的数据是最新版本；
- ✓ 安全性（安全攻击、数据的异地/国存储）。

7.8 列式数据库

最早的商业列式数据库是在1995年发布的Sybase IQ，但是一直到1999年左右才慢慢稳定到能够投入生产环境。

现在的大多数分析型数据库都是在2003-2005年从PostgreSQL分支出来的。其中，尤以Vertica为代表的列式数据库已在大规模**数据仓库环境**中证明其特别为数据仓库环境设计的思路在一些领域具有竞争优势。

列式数据库最大的优势是高效的储存空间利用率。普通的行式数据库一般压缩率在3:1到5:1左右，而列式数据库的压缩率一般在8:1到30:1左右。

代表性产品：Cassandra, Hbase, Vertica, ……

7.9 NoSQL

(1) 出现背景

SQL数据库在应对数据集群方面有较大的性能瓶颈。NoSQL（Not only SQL）概念最早于1998年提出。

(2) 最早的NoSQL数据库

Bigtable(Google)、Dynamo等。

(3) NoSQL数据库的特点(Adewale)

- ✓ 不使用关系数据模型，不使用结构化查询语言；
- ✓ 针对大型集群设计，可存储PB级别的数据，并处理大量的分析数据；
- ✓ 没有固定的模式，一条记录中可以存储任意数据。可应对不同类型的数据，比如文档、图片、视频等数据。

(4) 代表性产品

Bigtable(Google) 、 Dynamo 、 MongoDB 、 RavenDB 、 HBase(Apache)、 DeNSo DB、Oracle NoSQL 2.0、CouchDB、Redis、Cassandra 等。

7.10 NewSQL

(1) 概述

NewSQL由Matthew Aslett提出，是对各种新的可扩展/高性能数据库的简称，这类数据库不仅具有NoSQL对海量数据的存储管理能力，还保持了传统数据库支持ACID和SQL等特性。

(2) 代表性产品

Clustrix、GenieDB、ScalArc、Schooner、VoltDB、RethinkDB、ScaleDB、Akiban、CodeFutures、ScaleBase、Translattice、NimbusDB、TiDB、Drizzle、带有NDB的MySQL集群和带有HandlerSocket的MySQL。

1.2 数据库系统中的基本概念

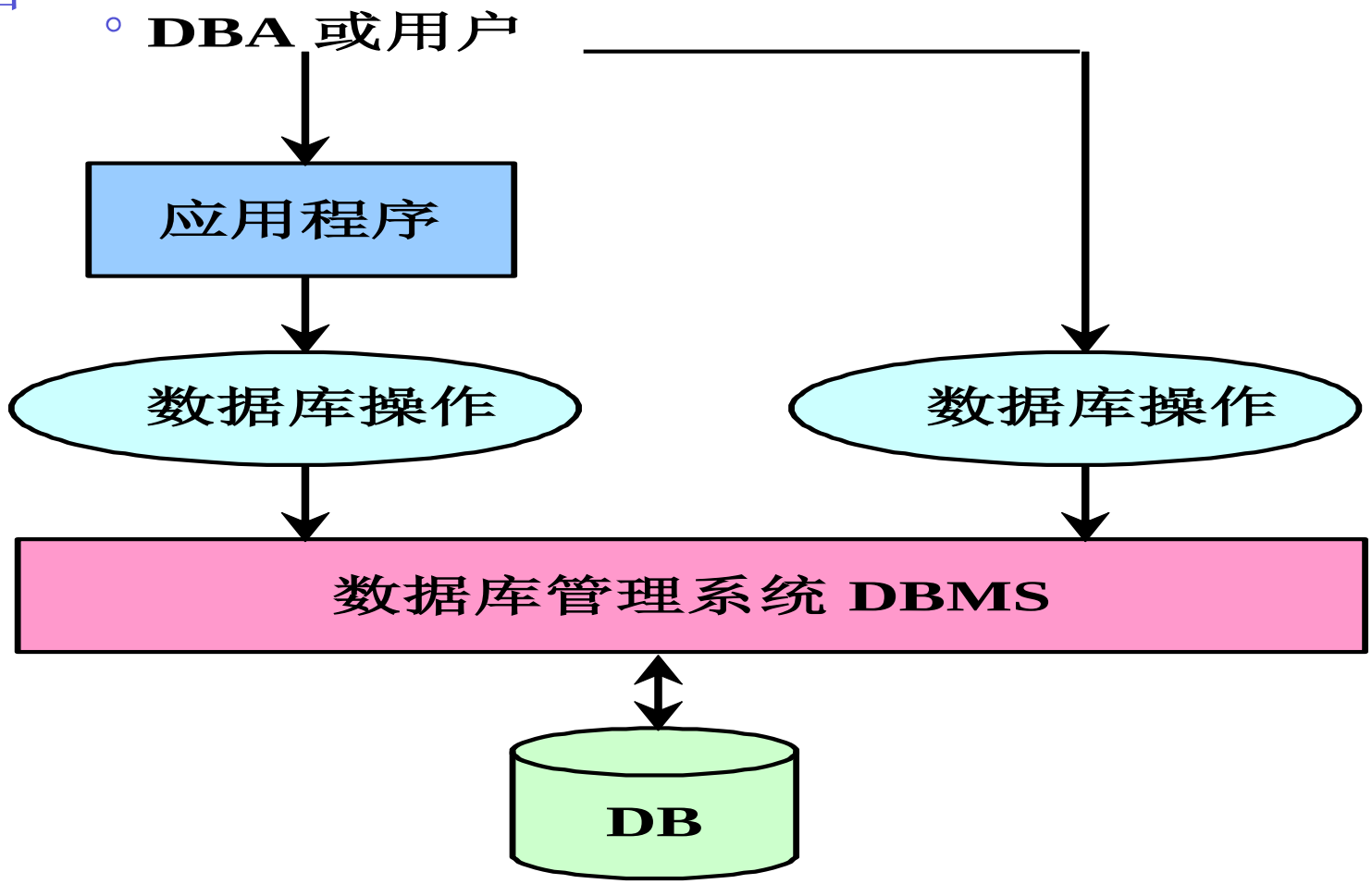
一. 数据、数据库、数据库管理系统和数据库系统

数据(Data) 是描述现实世界中各种具体事物或抽象概念的可存贮并具有明确意义的信息。

数据库(Database, DB) 是相互关联的数据集合。

数据库管理系统(Database Management System, DBMS) 是一个通用的软件系统，由一组计算机程序构成。它能对数据库进行有效的管理，包括存储管理、安全性管理、完整性管理等；同时，它也为用户提供了一个软件环境，使其能够方便快速地创建、维护、检索、存取和处理数据库中的信息。

数据库系统(Database System, DBS) 由数据库和数据库管理系统构成，更广义的构成则为“DB+DBMS+数据库管理员（DataBase Administrator, DBA）+应用程序+用户”。



数据库系统环境示意图

数据字典(Data Dictionary, DD) 是数据库系统中的一个特殊文件，用于存储数据库的一些说明信息，这些说明信息称为**元数据(Meta Data)**。

数据库操作(Database Operation) 在数据库应用中，最常见的数据库操作有：增加、删除、修改和查询。

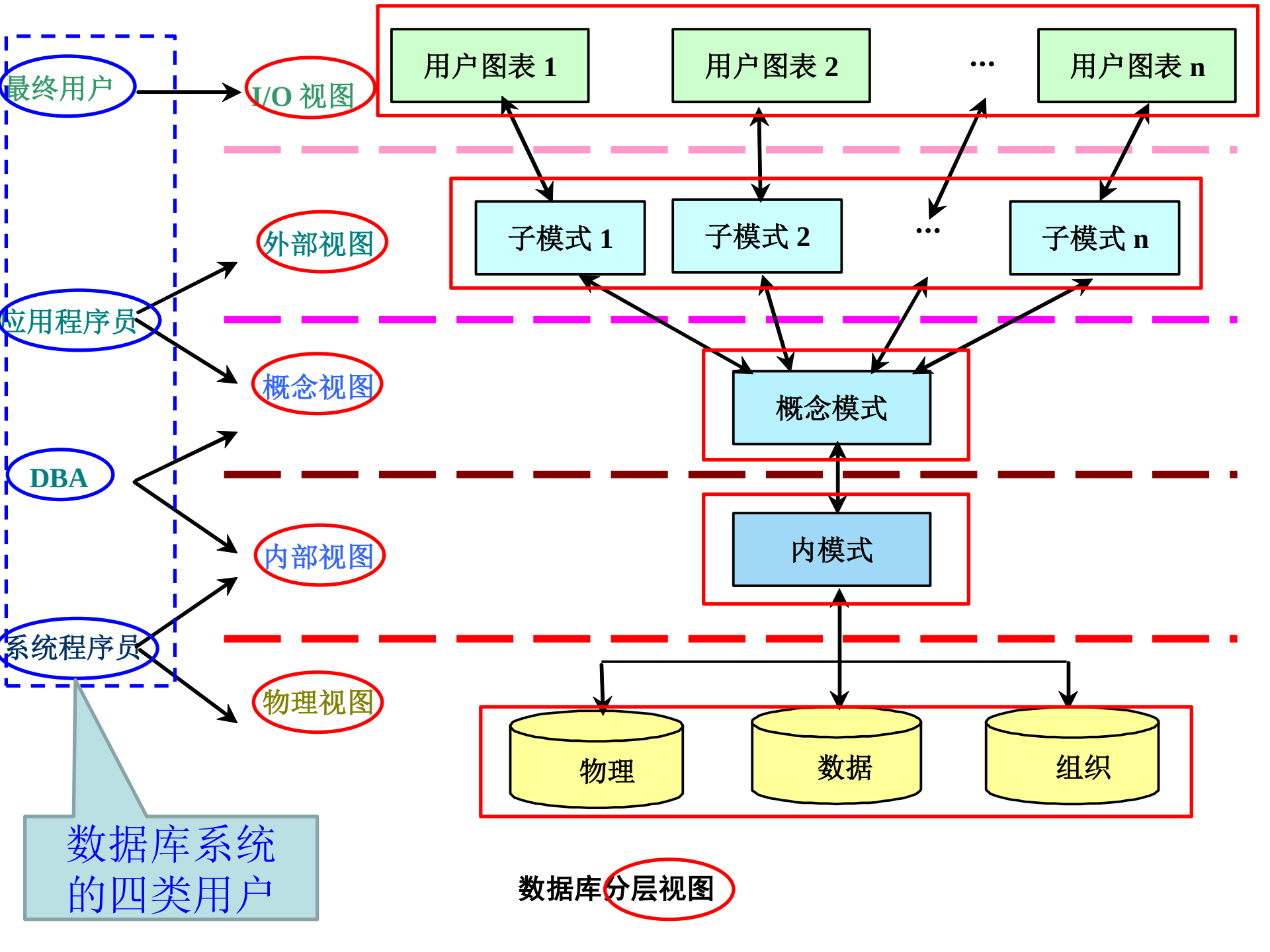
大型数据库与微机数据库区别

二. 视图

视图(View) 对同一数据库的每一种理解即被称为该数据库的一个视图。

数据库的分层视图

视图原始定义扩展之二：一个用户→不同层次用户


最终用户
I/O 视图
用户图表 1
用户图表 2
…
用户图表 n
外部视图
子模式 1
子模式 2
…
子模式 n
应用程序员
概念视图
概念模式
DBA
内部视图
内模式
系统程序员
物理视图
物理
数据
组织
数据库系统
的四类用户
数据库分层视图

输入输出数据视图 即终端用户所见到的输入输出数据结构描述，也就是最终用户所见到的数据库的样子。

外部视图(External View) 局部数据库逻辑结构称为外部视图。这种视图在数据库设计时通常以图形的形式（如E-R图）表示，有的又叫视图或用户视图。

概念视图(Conceptual View) 整个数据库系统的全局逻辑结构。这种逻辑结构称为概念模型，它不包含任何数据库的实现细节，如：何种DBMS、文件组织、存取方法等。这种逻辑结构的形式化描述称为概念视图。在数据库设计时，概念视图通常也以E-R图表示。

内部视图(Internal View)或存储视图特定的DBMS所处理的数据库的内部结构称为内部模型，其形式化描述称为内部视图或存储视图，它将数据库表示为”内部记录”或”存储记录”的集合。存储记录仍然是逻辑性的，它不存储设备上的物理记录或物理块，也不涉及任何具体设备限制，如：柱面或磁道的大小等，所以存储视图还不是最底层的物理层。存储视图还指明存储记录的物理顺序、以及它们如何彼此关联。存储视图的语言形式定义称为内部模式。

物理视图(Physical View) 数据库在存储设备上的物理组织称为物理模型，其描述称为物理视图。它包含了所使用设备特征、物理记录或块的组成、寻址技术和压缩存储技术等的说明。

三. 数据抽象、数据模型与数据模式之关系

数据抽象(Data Abstraction) 即是将数据抽象化、逻辑化，是对数据的抽取过程。在数据库系统中，有三种级别的数据抽象，即：视图级抽象、概念级抽象和物理级抽象。

数据模型(Data Model) 即是对数据进行抽象化表示的工具，主要使用逻辑概念（如对象、对象属性、对象联系等）来表示数据。

由于抽象级别的存在，数据模型也存在相应的级别。如：概念数据模型、逻辑数据模型、物理数据模型等。对于抽象级别高的概念数据模型我们叫它**语义(Semantic)数据模型**，如ER模型。

数据模式(Data Schema) 利用数据模型组织抽取的数据所得的结果，也即是数据抽象的结果。

四. 数据模型（Data model）

1. 数据模型的三个评价标准

通俗来讲，数据模型就是对现实世界的模拟、描述或表示。数据模型应满足的三个要求：

（1）比较真实地描述现实世界；

（2）易为用户所理解；

（3）易于在计算机上实现。

这两个要求是矛盾的，因此产生了高级模型（面向人）和低级模型（面向数据库）

2. 数据模型的三要素

（1）数据结构

用于描述系统的静态特性。数据结构不仅要描述数据（或对象）本身，还要描述数据（或对象）之间的联系。

（2）数据操作

用于描述系统的动态特性。包括操作及有关的操作规则。数据库的主要操作有：插入、删除、修改和查询。

注：ER模型作为一种高级数据模型，没有数据操作。

（3）数据的约束条件

是一组完整性规则的集合。完整性规则是数据模型中数据及其联系所具有的约束规则，用来限定数据库状态以及状态的变化，以保证数据的合理。

3. 数据模型应用现状

为何要使用多种数据模型（高级\低级数据模型）？

(1) 现实管理系统的用户与计算机管理系统的设计人员之间的专业差异。

(2) 用户理解与计算机实现的矛盾。

4. 数据模型的分类

视图抽象与概念抽象阶段——**高级**数据模型（如：ERM），也称概念数据模型或语义数据模型。

物理抽象阶段——**低级**数据模型（如：关系/层次/网状模型——**3个**传统数据模型），也叫逻辑数据模型。

数据模型分类

- 概念数据模型、语义数据模型、高级数据模型：ERM（实体-联系模型）
- 逻辑数据模型、低级数据模型、传统数据模型：关系/层次/网状模型
- 物理数据模型：数据的物理存储结构，只有DBMS本身的开发人员关心，DBS开发人员不用关心

5. 数据模式（Data schema）

抽取的数据用数据模型组织后，得到的结果就是数据模式。也就是说，数据模式是所抽取数据的表现形式，是数据抽象的结果。

狭义上，可以把数据模型、数据模式与C语言中的结构体（struct）进行类比：

结构类型 → 数据模型（数据抽象的工具）

具体的某个结构 → 数据模式（数据抽象的结果）

数据文件 → 数据库（数据的物理存储）

6. 传统数据模型回顾

（1）层次数据模型(Hierarchical Data Model)

HDM是数据库系统中最早出现的数据模型。60年代后期，IBM开发出IMS (Information Management System) DBMS，是层次数据库管理系统的典型代表。

用树形结构表示各类实体以及实体之间的联系。

现实世界中许多实体之间的联系就呈现出一种很自然的层次关系，如：行政机构、家庭关系等。层次模型是以记录型为结点的有向树。

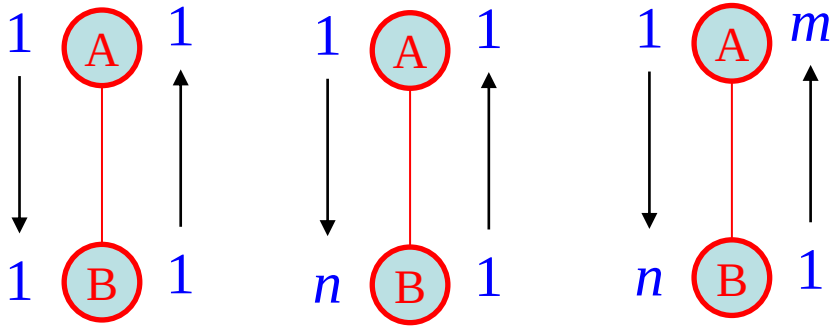
层次数据模型

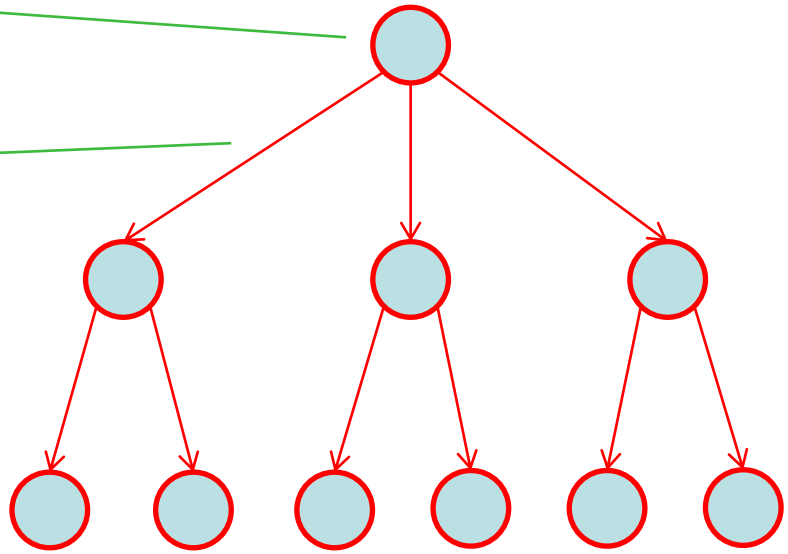
结构
- 对象
- 对象间联系

操作 ： 增删改查

约束

1:1 ✓　　1:*n*/*n*:1 ✓　　*m*:*n* ✗（层次模型不支持 m:n 联系）





按树的定义，层次模型有以下两个限制：

☆只有一个结点没有双亲结点，即根结点；

☆根以外的其他结点有且只有一个双亲结点。

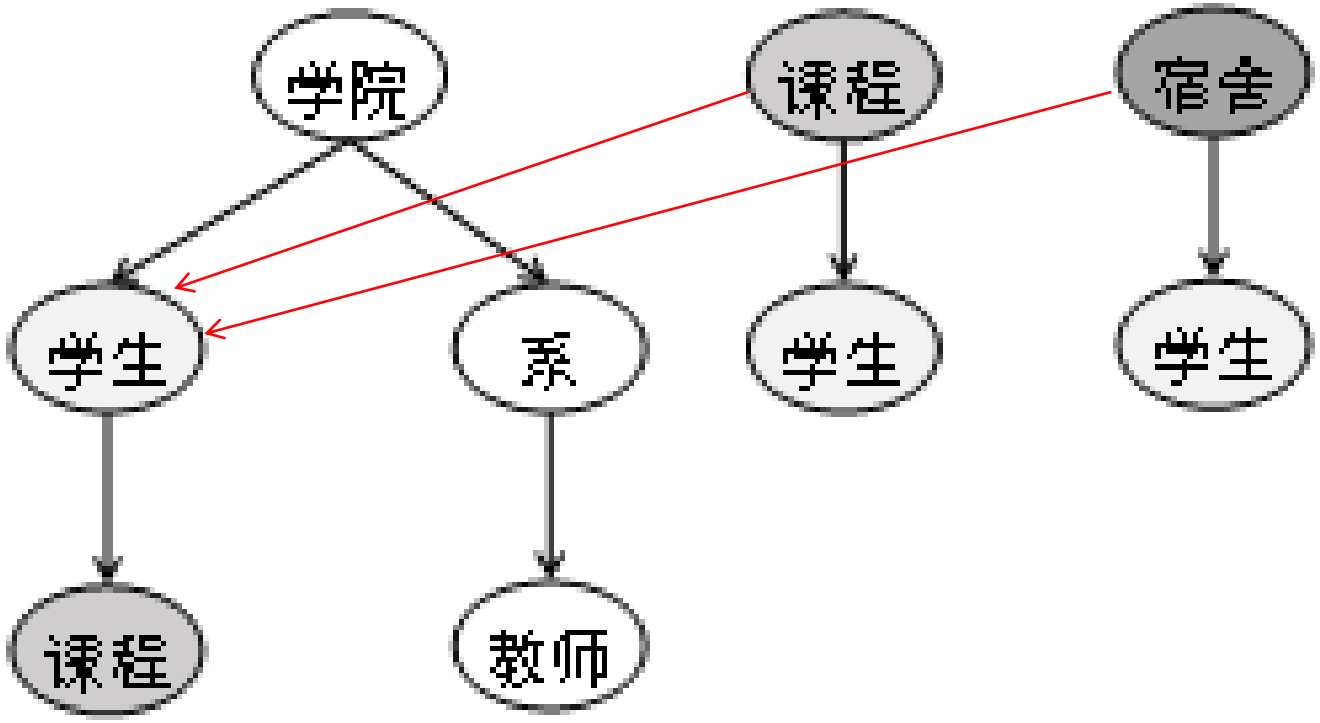


图 1-15 层次模型示例

层次模型中，每个结点表示一个记录类型，结点之间的连线表示记录类型间的联系。这种联系只能是父子联系。

层次模型的另一个最基本的特点是：任何一个给定的的记录值只有按其路径查看时，才能显出它的全部意义，没有一个子女记录值能够脱离双亲记录值而独立存在。

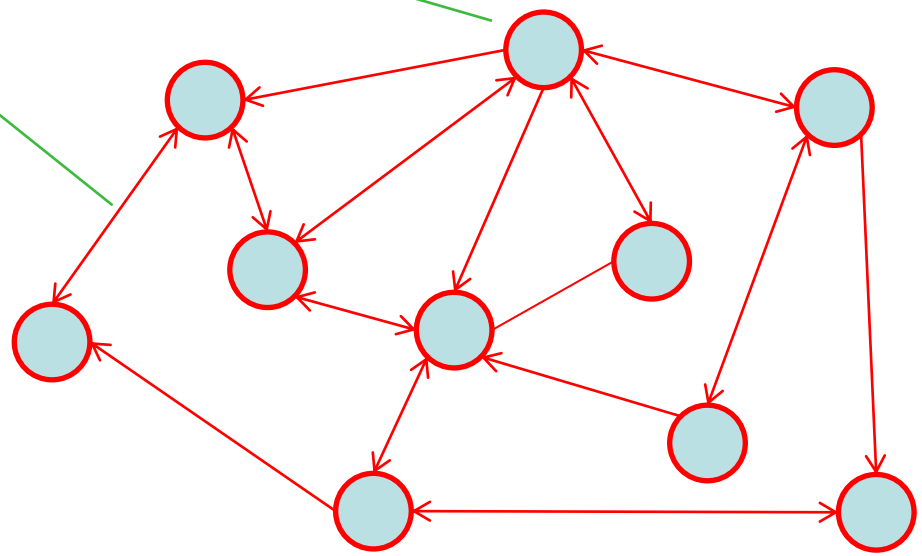
（2）网状数据模型(Net Data Model)

用层次模型表示非树型结构很不直接，网状模型则可克服这一弊病。

网状数据模型的典型代表是DBTG系统，亦称CODASYL系统。是由美国数据库系统语言协会CODASYL (Conference On Data System Language)下属的DBTG (DataBase Task Group)于60年代末70年代初提出的一个系统方案，形成DBTG报告。

网状数据模型

- 结构
  - 对象
  - 对象间联系
- 操作 ：增删改查
- 约束
  - 1:1 ✓
  - 1:n/n:1 ✓
  - m:n ✓



网状模型比层次模型更具普遍性。

它去掉了层次模型的两个限制，允许结点有多个双亲结点。可比层次模型更直接地描述现实世界。

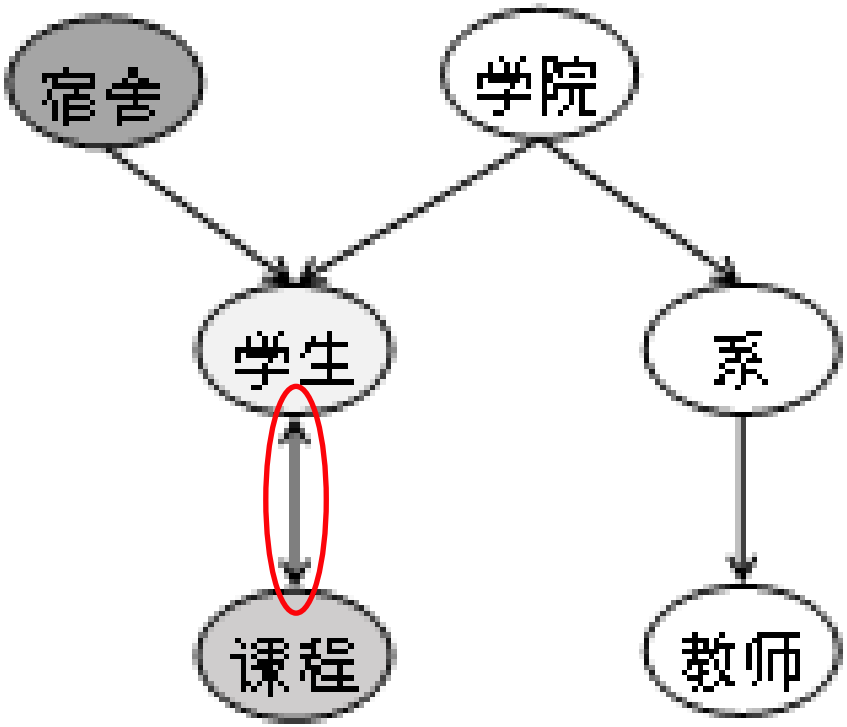


图 1-17 网状模型示例

五.数据库语言

数据库语言是DBMS提供给用户定义结构、操纵数据和管理DBMS的一个界面，是支持数据独立性的重要因素。

数据库语言分为：

数据定义子语言(Data Definition Language, DDL) 用来定义数据库模式。

数据操纵子语言(Data Manipulation Language, DML) 用来表示用户对数据库的操作请求（增加、删除、修改、查询）。由于数据库语言其主要的功能是查询数据库中的信息，故经常称之为**数据查询语言**。

数据控制子语言(Data Control Language, DCL) 用来定义数据库数据库用户、为用户授权、设置系统参数和调整系统性能等。

六. 关系数据库语言—SQL语言

SQL(Structured Query Language,结构化查询语言)是一种集数据定义、数据操纵和数据控制子语言为一体的、标准化（80年代后期）的、**关系型数据库**语言。

目前应用最普遍的标准化版本为SQL-92，被ANSI(American National Standards Institute)、ISO(International Standards Organization)采纳。

SQL语言的第一个版本是由IBM公司SAN JOSE实验室为SYSTEM R关系数据库管理系统设计的。

SQL语言既可以作为交互式（Interactive）数据库语言使用，也可以嵌入（Embedded）到程序设计语言中作为其子语言使用，这时前者称为宿主语言(Host Language)，如：C/C++语言、PowerBuilder、Delphi等。

SQL 标准有：SQL-86、SQL-89、SQL-92、SQL:1999、SQL:2003/2006/2008/2011/2016/2023。其中，SQL-86和SQL-89属SQL1（表示SQL版本1），SQL-92属SQL2，而SQL:1999则属SQL3。

注意：不同的数据模型，其语言是不一样的。

如 MongoDB 的数据模型是文档模型（document Model），数据以 JSON-like 的二进制格式（BSON）存储，如：

{ "_id": ObjectId("5f9d1b3b0c9d6e1a1c2b4567"), "name": "Alice", "age": 30, "hobbies": ["reading", "hiking"], "address": { "city": "Beijing", "street": "Xidan" }, "scores": [90, 85, 92] }

其查询语句是基于 BSON 的查询系统，如：

```
// 查询年龄大于 25 岁的用户
db.users.find({ age: { $gt: 25 } });
// 查询名字为 "Alice" 或年龄小于 30 的用户
db.users.find({ $or: [{ name: "Alice" }, { age: { $lt: 30 } }] });
// 查询名字不为空的用户
db.users.find({ name: { $ne: null } });
```